Paper 2 / Algorithms and Programming

OCR H446/02 / Specification order

Choose a topic. Page numbers match the PDF viewer.

2.1 / Elements of computational thinking

2.1.1-2.1.4 / Abstractly, ahead, procedurally and logically **2**

2.1.5 / Thinking concurrently **15**

2.2.1 / Programming techniques

2.2.1(a) / Constructs and integrated programming practice **21**

2.2.1(b) / Recursion and the call stack **42**

2.2.1(c) / Global and local scope **51**

2.2.1(d) / Subroutines, parameters, files and modularity **55**

2.2.1(e) / Debugging with an IDE **70**

2.2.1(f) / Object-oriented programming **77**

2.2.2 / Computational methods

2.2.2 / Computational methods and worked applications **83**

2.3.1 / Algorithms

2.3.1(a)-(d) / Algorithm design, tracing and complexity **103**

2.3.1(e) / Algorithms for data structures and trees **127**

2.3.1(f) / Searching and sorting **148**

2.3.1(f) / Dijkstra and A* pathfinding **165**

Mixed revision

Mixed memory banks **176**

Worked examples, practice and answers stay within each topic.

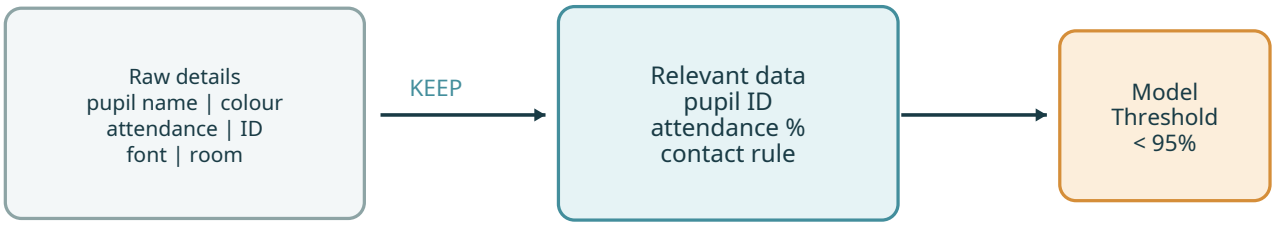
2.1.1 / Thinking abstractly

Models deliberately ignore some details; justify what stays and what goes.

OCR 2.1.1 • SPECIFICATION CHECK

- Distinguish abstraction, data abstraction and decomposition.
- Create an appropriate abstraction for a scenario and explain excluded details.
- Evaluate whether a graph, model or abstract data type captures the problem.

A school attendance scenario



To identify an attendance alert, the model needs an identifier, an attendance figure and a defined alert rule. Hair colour and display font are irrelevant to that calculation. The model would need additional fields to support a different purpose.

Term	Precise explanation
Abstraction	Retain relevant properties and disregard unnecessary details for a specified purpose.
Data abstraction	Describe a data type through its values and operations, rather than all implementation details.
Graph abstraction	Represent relevant objects as vertices and relationships as edges.
Decomposition	Divide the overall process into subtasks; different from removing details.

Exam trap: “ignore details” alone is insufficient. Name what information is kept, what is omitted and why this is appropriate in the scenario.

Theory / Key terminology

Use these exam-accurate terms when explaining algorithms and programming.

Abstraction	Removing unnecessary detail while retaining what is important to a problem.
Data abstraction	Representing data through an appropriate interface or model.
Decomposition	Breaking a complex problem into manageable smaller problems.
Thinking ahead	Anticipating inputs, outputs, constraints and consequences.
Procedural thinking	Organising a solution into a clear sequence of steps.
Logical thinking	Using evidence, conditions and rules to draw valid conclusions.
Precondition	Condition that must be true before an algorithm or operation starts.
Model	A simplified representation used to reason about a real system.

WORKED EXAMPLE 1 → WORKED EXAMPLE 2 / building an abstraction

WORKED EXAMPLE 1 / DESIGN A LIBRARY LOAN MODEL

Question	Worked response
Purpose	Track whether a borrowed book is overdue.
Keep	Loan ID, borrower ID, due date, return date (if returned).
Ignore	Book cover colour and shelf decoration: neither changes the overdue rule.
Rule	Overdue when not returned and current date is later than due date.

WORKED EXAMPLE 2 / new school scenario

Model a school-bus route planner that aims to minimise journey distance. State THREE relevant data items, TWO details to ignore and ONE limitation of your abstraction.

_____

_____

_____

_____

Guided check: Relevant: stops, connections, distances (and possibly timetable for a time-based goal). Ignore bus paint colour, driver name. A distance-only model does not account for traffic or road closures.

Thinking abstractly · independent exam questions

Q1 [4 marks] Distinguish abstraction from decomposition, with a game example.

DIAGRAM / EXPLANATION

Q2 [7 marks] An app gives wheelchair-accessible routes across a school. Propose FOUR essential model features and TWO features to exclude. Explain one limitation.

Thinking abstractly · independent exam questions

Q3 [3 marks] Describe how a graph could represent bus stops and connections.

Q4 [3 marks] Challenge: why could ignoring stairs be acceptable for one model but unsafe for another?

Abstraction: marking points

Check only after completing the independent questions.

Q1 · 4 marks — Abstraction versus decomposition

- Abstraction removes a detail that does not affect the model’s purpose. [1]
- For example, exclude background animation when modelling a game’s score. [1]
- Decomposition splits the overall program into smaller tasks. [1]
- For example, separate input, scoring and collision detection. [1]

Q2 · 7 marks — Accessible school routes

- Retain step-free path connections. [1]
- Retain locations of accessible entrances. [1]
- Retain locations of usable lifts or ramps. [1]
- Retain sufficient route connectivity or accessible links between buildings. [1]
- Exclude wall colour when it does not affect accessibility. [1]
- Exclude irrelevant details such as locker numbers. [1]
- A lift closure or temporary obstruction could make the model’s recommendation incorrect. [1]

Q3 · 3 marks — Bus network as a graph

- Vertices represent the bus stops. [1]
- Edges represent direct bus connections. [1]
- Weights could show journey time or distance. [1]

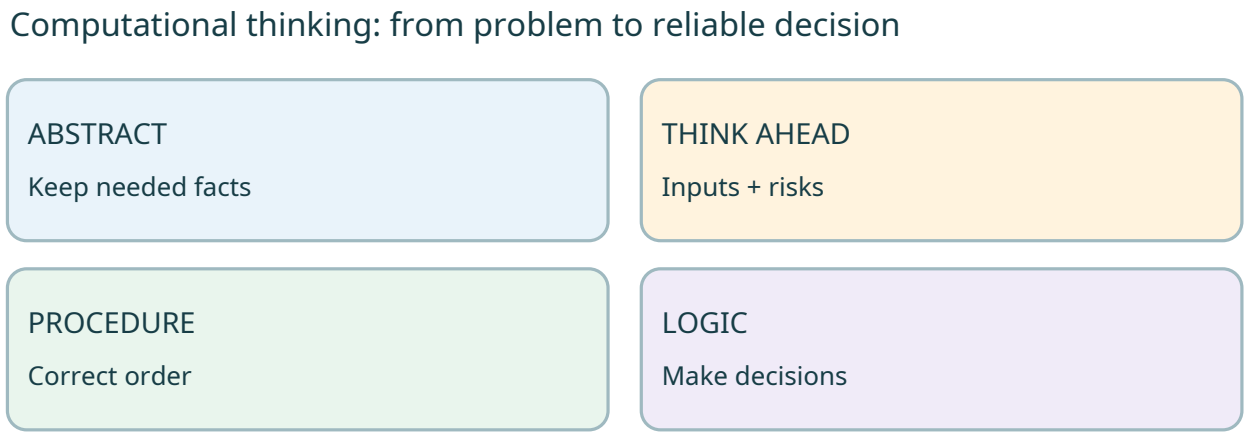
Q4 · 3 marks — When stairs matter

- Stairs can be ignored when irrelevant to a model’s purpose, such as a bus timetable. [1]
- For wheelchair-accessible routing, missing stair information could suggest an impossible route. [1]
- The safe level of abstraction depends on the model’s purpose and intended users. [1]

Equivalent technically correct responses can earn credit where they meet the stated criteria.

2.1.1–2.1.4 Thinking like a computer scientist

Remember four moves: KEEP → PREPARE → ORDER → DECIDE. These are memory prompts for thinking abstractly, ahead, procedurally and logically; the examination names remain essential.



NOW BE PRECISE

Abstractly: retain details relevant to a particular purpose. Ahead: identify inputs, outputs, preconditions, caching and reuse. Procedurally: decompose the solution and respect dependencies. Logically: identify decision points, conditions and resulting program flow.

WORKED EXAMPLE 1 / Map model

To find the bus route with the fewest stops, model stops as vertices and connections as edges. The colour of the buses can be omitted. If the task changes to fastest route, travel-time weights become essential.

WORKED EXAMPLE 2 / School timetable

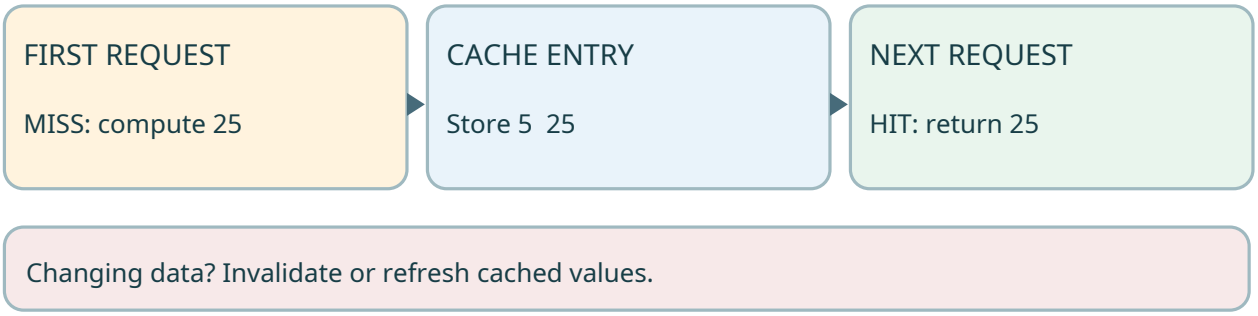
A timetable clash checker stores room identifiers, start times and end times. What can it omit, and what condition indicates a clash?

Guided response

Ignore decorations and desk colours. A clash occurs if the room is the same and the two time intervals overlap. A time interval must have a clearly defined start and end.

Thinking ahead: a cache is a shortcut

A cache saves repeated work - but can become stale



Caching can avoid repeated work. The first request for the square of 5 computes 25 and stores it; another request can reuse the answer. Cache entries consume memory. If the underlying information changes, they can become stale.

WORKED EXAMPLE 1 / Cached price

A ticket price of £3.40 is stored under zone A. A second query for A is a cache hit. If the price changes, the old cached price must be invalidated, refreshed or expired.

WORKED EXAMPLE 2 / Attendance count

A live attendance count changes from 12 to 15 while the cache still holds 12. Explain the problem and one suitable correction.

Guided response

The cached 12 is now incorrect. Invalidate/update it when the attendance record changes, or use a justified expiry rule. Cache correctness matters as much as speed.

Procedures and decisions

A program's modules must follow its dependencies



Sending confirmation before a successful save is a logic fault.

For an online booking process, validate → check availability → save → confirm. Sending confirmation before the booking is stored is a dependency error. Each stage can become a small testable subprocedure.

WORKED EXAMPLE 1 / Logic

A cinema admits customers only if age >= 15 AND hasTicket. If age=16 but hasTicket=false, the AND expression is false and entry is refused.

WORKED EXAMPLE 2 / Independent task

A reward unlocks if score >= 80 OR teacherOverride=true. Evaluate score=72, teacherOverride=true.

Guided response

72 >= 80 is false; teacherOverride is true; false OR true is true. Explain each Boolean result before describing the branch taken.

COMMON MISTAKE

Abstraction is not simply “removing detail”. It means selecting the right details for the purpose. Likewise, thinking ahead is not just predicting: it includes preconditions, input/output contracts, caching and reuse.

Exam Practice

Q1 · 4 marks For a bus route-finding model, identify two details to retain and two to omit when optimising the fewest stops.

Q2 · 4 marks Give the input and output of a function that returns the maximum of a nonempty integer array, and explain one precondition and why it matters.

Q3 · 6 marks A school caches room-availability results. Explain two benefits, two limitations and one way to prevent stale results after a booking.

Q4 · 6 marks Design the correct order of subprocedures for reserving concert tickets and explain two important dependencies.

Q5 · 6 marks A pupil receives a 10% discount when member=true AND basket>=50. Trace the decision for (true,60) and (false,60), then explain what would change if AND were replaced by OR.

Model answers: computational thinking

Each mark must relate to what the question actually requests. Accept appropriate alternatives.

Q1 · 4 marks

1. Retain the bus-stop identifiers/locations.
2. Retain permitted connections between stops.
3. Omit a detail irrelevant to the number of stops, such as bus colour.
4. Omit another irrelevant detail, such as seat fabric or driver’s favourite food.

Q2 · 4 marks

1. Input: an integer array.
2. Output: the greatest element of that array.
3. The precondition is that the array is non-empty.
4. An empty array has no first element, so initialising the maximum from a[0] would be invalid.

Q3 · 6 marks

1. Cached results can be returned faster.
2. Cache hits reduce repeated database or calculation work.
3. Keeping cached results uses extra memory/storage.
4. Cached availability can become stale after another booking.
5. Invalidate/update the relevant cached result when the booking succeeds.
6. The next request can then fetch/compute the current availability rather than reuse stale data.

Q4 · 6 marks

1. ValidateRequest checks the input.
2. CheckSeats verifies availability.
3. SaveBooking records the transaction.
4. SendConfirmation notifies the customer.
5. Availability must be checked before attempting to save.
6. Confirmation must follow a successfully committed save.

Q5 · 6 marks

1. (true, 60): membership is true.
2. Both conditions pass; discount gives **54**.
3. (false, 60): the membership condition is false.
4. AND fails; price remains **60**.
5. With OR, a non-member spending 60 wrongly receives the discount.
6. With OR, a member spending less than 50 would also wrongly receive the discount.

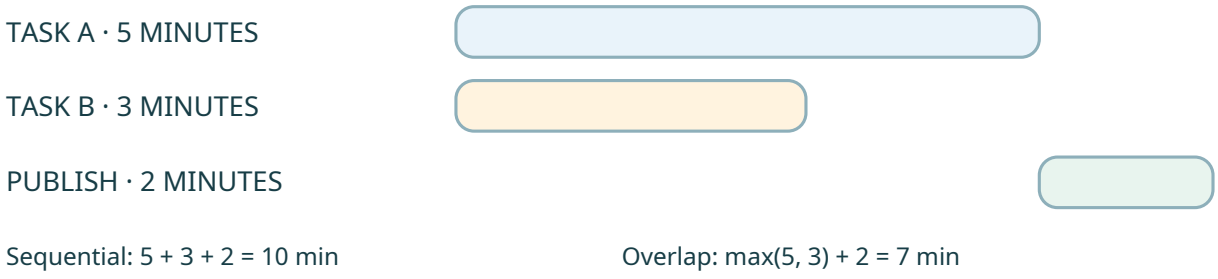
2.1.5 Thinking concurrently

Remember two cooks, one kitchen. Two cooks can prepare separate ingredients at the same time, but the shared oven may become a bottleneck. Likewise, a computer must identify independent tasks, dependencies and shared resources.

NOW BE PRECISE

Concurrent tasks make progress during overlapping time intervals; execution may be interleaved on one processor. Parallel processing means operations are actually executed at the same time using separate processing resources. Concurrent design does not guarantee a speed-up.

Two tasks can overlap; the final task must wait



WORKED EXAMPLE 1 / Two independent checks

Data checks A and B take 5 and 3 minutes. Publishing takes a further 2 minutes and must wait for both checks. Sequentially: 5 + 3 + 2 = 10 minutes. In the ideal overlapping case: max(5,3) + 2 = 7 minutes.

WORKED EXAMPLE 2 / Website release

Preparing images takes 4 minutes; creating CSS takes 6 minutes; deployment takes 1 minute and waits for both. Work out both totals.

Guided working

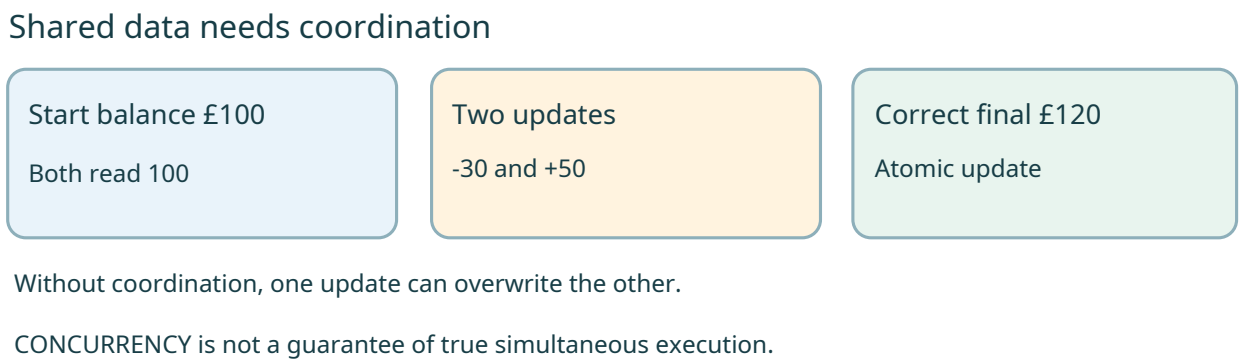
Sequential = 4 + 6 + 1 = 11 minutes. Ideally concurrent = max(4, 6) + 1 = 7 minutes. The two preparations are independent; deployment is dependent.

Theory / Key terminology

Use these exam-accurate terms when explaining algorithms and programming.

Concurrency	Multiple tasks make progress over an overlapping time period.
Parallelism	Tasks execute at the same time on separate processing resources.
Race condition	Outcome varies depending on the timing or ordering of operations.
Critical section	Code accessing shared state that requires controlled entry.
Mutual exclusion	Ensuring only one task accesses a protected resource at a time.
Deadlock	Two or more tasks remain blocked waiting for resources held by others.
Synchronisation	Coordinating tasks so shared state remains consistent.

Dependencies and shared state



If two processes read the same starting bank balance of £100, one withdrawing £30 and one depositing £50, careless overlapping updates may overwrite each other. A correct coordinated outcome is £120.

COMMON MISCONCEPTION

Doing tasks concurrently is not always faster. Waiting for shared data, locks, network latency, synchronisation or sequential dependencies can dominate the runtime. A race condition arises when an outcome incorrectly depends on the order/timing of unsynchronised access.

WORKED EXAMPLE 1 / Safe balance update

A safe system coordinates the update to a shared bank balance, for example using a suitable transaction/lock, so one update does not accidentally overwrite the other.

WORKED EXAMPLE 2 / Report publishing

Two independent report sections are finished at different times. Explain why publication must wait for both, and give one possible coordination cost.

Guided explanation

Publication has a dependency on both sections. The final task cannot start until both are complete. Coordination, data transfer and waiting for a shared resource can reduce any ideal speed-up.

Exam practice / Concurrent thinking

Q1 [4 marks] Independent tasks A and B take 3 and 6 minutes. A final task C takes 2 minutes and depends on both. Calculate sequential and ideal overlapping completion times. Show working.

Q2 [4 marks] Explain the difference between concurrency and parallel processing, giving one example of each.

Q3 [6 marks] Two banking operations read and update the same account balance. Explain how a race condition can occur and how to prevent an incorrect balance.

Q4 [6 marks] Tasks A (4 min) and B (7 min) are independent. C (2 min) requires A and B, and D (1 min) requires C. Calculate the shortest ideal completion time and evaluate two reasons why the real time may be longer.

Model answers: concurrent thinking

Show the method for the calculations. Award each point separately.

Q1 · 4 marks

1. For sequential execution, add **3 + 6 + 2**.
2. Sequential total = **11 minutes**.
3. For ideal overlap, use **max(3, 6) + 2** because C must wait for both.
4. Ideal overlapping total = **8 minutes**.

Q2 · 4 marks

1. Concurrency allows tasks to make progress over overlapping periods, possibly interleaved.
2. Example: one CPU core time-slices between two tasks.
3. Parallelism means operations execute simultaneously.
4. Example: separate CPU cores execute independent tasks at the same time.

Q3 · 6 marks

1. Both operations can read the same initial balance.
2. Each calculates an updated balance from that stale value.
3. One write can overwrite the other.
4. This causes a lost update or incorrect final balance.
5. Use an appropriate lock, serialisable transaction or atomic update.
6. Ensure changes are applied consistently without losing either valid operation.

Q4 · 6 marks

1. A and B can run at the same time.
2. Both finish after **max(4, 7) = 7 minutes**.
3. C then needs 2 minutes, followed by D taking 1 minute.
4. Ideal total = **7 + 2 + 1 = 10 minutes**.
5. Resource contention or waiting for locks can increase the real time.
6. Scheduling, communication or synchronisation overhead can also increase it.

2.2.1(A,C,D) / SIS: constructs, scope and modularity

Remember: SIS: Sequence, Iteration, Selection.

SIS: THE THREE PROGRAMMING CONSTRUCTS

S	I	S
SEQUENCE	ITERATION	SELECTION
in order	repeat	choose

OCR calls selection "branching". SIS is a memory hook, not official OCR wording.

ANALOGY - PICTURE IT

Making tea involves steps in order (sequence), repeated stirring (iteration) and choosing milk according to a condition (selection).

NOW BE PRECISE - WHERE THE ANALOGY STOPS

OCR uses the term branching for selection. Program flow depends on the exact condition and iteration termination, not everyday intuition.

THEORY / KEY TERMINOLOGY

Sequence	Instructions execute in a defined order.
Iteration	Repeats an instruction block while a condition or count determines repetition.
Selection / branching	Executes one of several paths depending on a Boolean condition.
Local variable	Declared within a limited scope such as a function.
Global variable	Defined with wider program scope; may complicate state changes.
By value vs by reference	By value passes an independent value; by reference permits changes to the caller variable in the relevant language.

SIS: constructs, scope and modularity / understand and model

Students first code a simple sequence, then notice repetition, then introduce a loop, and finally conditionally choose actions. A procedure performs a reusable task; a function returns a value. Modularity reduces duplication and supports testing. Pass-by-value and pass-by-reference are distinct semantics, and C# ref parameters require appropriate syntax in both the method and the call. Beware: with reference-type objects, passing a reference by value is not the same as passing the variable by ref.

WORKED EXAMPLE 1 / worked example

Code first: int total=0; for (int n=1; n<=5; n++) { if (n % 2 == 0) total += n; }

Sequence: initialise total to 0 before the loop.

Iteration: n takes 1, 2, 3, 4, 5.

Selection: only even n values are added: 2 + 4 = 6.

WORKED EXAMPLE 2 / same skill, different problem

YOUR TURN

Write C# to add only odd values from 1 through 4 inclusive, then trace the final total and identify all three SIS constructs.

Scaffold: Start with total=0. Use a FOR loop and an IF condition testing n % 2 != 0.

My working:

GUIDED CHECK - READ AFTER ATTEMPTING

Sequence: Initialise total, then output the final value.
Iteration: n runs from 1 through 4.
Selection: Odd values 1 and 3; total = 4.

SIS: constructs, scope and modularity / practice

Q1 [4 marks] State the three constructs in SIS and give the OCR term for selection.

Q2 [3 marks] A loop runs for i=1 to 5 inclusive; it adds i when i%2==0. What is total? Show contributions.

Q3 [3 marks] Explain why a function is suitable for calculating the average mark from an array.

SIS: constructs, scope and modularity / practice

Q4 [4 marks] Distinguish C# parameter passing by value and by ref for an int variable.

Check your answers — SIS

Award marks for the required distinct facts or reasoning. Equivalent accurate wording is acceptable.

Q1 · 4 marks · Programming constructs

- Sequence: instructions execute in order. [1]
- Iteration: a block of instructions repeats. [1]
- Selection: a condition chooses a path. [1]
- OCR also calls selection branching. [1]

Q2 · 3 marks · Loop result

- i = 2 satisfies the even condition and is added. [1]
- i = 4 also satisfies the condition and is added. [1]
- Total = 2 + 4 = 6. [1]

Q3 · 3 marks · Functions

- The average-calculation logic is kept in one subroutine. [1]
- It can be reused with different arrays or callers. [1]
- It returns the calculated average to the caller. [1]

Q4 · 4 marks · Value and reference

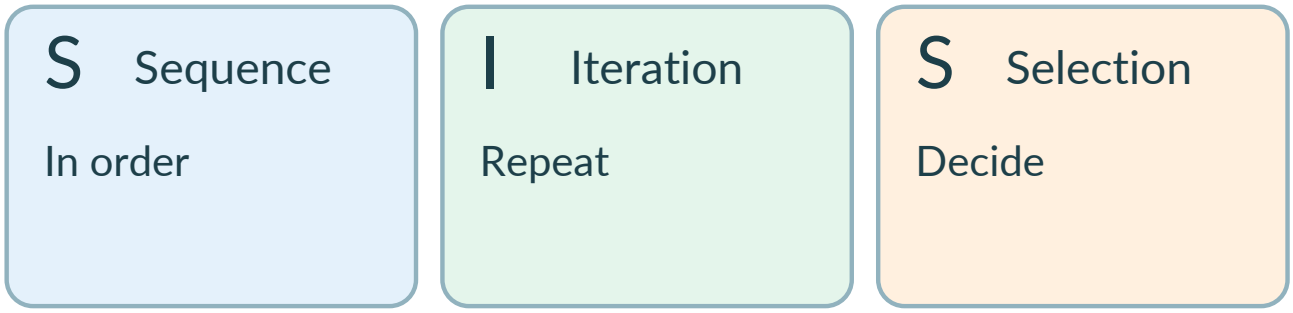
- A by-value int parameter initially gets a copy of the value. [1]
- Assigning to that copy does not update the caller variable. [1]
- A ref int parameter refers to the caller variable. [1]
- Changes through the ref parameter are visible to the caller. [1]

Remember

SIS = Sequence, Iteration, Selection. For OCR wording, use sequence, iteration and branching.

Code First: SIS in Action

Remember SIS: sequence executes statements in order, iteration repeats, and selection (OCR: branching) chooses a path. Every variable change should be traceable.



WORKED EXAMPLE 1 · Trace before you write

Predict the total after each iteration. Do not jump straight to the final number.

```
int score = 0;
for (int i = 1; i <= 4; i++) {
    if (i % 2 == 0) score += i * 2;
    else score += i;
}
Console.WriteLine(score);
```

Trace: i = 1 → 1; i = 2 → 5; i = 3 → 8; i = 4 → 16. The output is 16. The conditional chooses between two update expressions.

Remember this

A cooking recipe: follow a sequence, repeat a stirring action, then choose an ingredient based on a condition. Now be precise: iteration is controlled by a loop; selection is controlled by a Boolean condition.

WORKED EXAMPLE 2: a different trace

A temperature logger analyses readings. Predict both outputs before inspecting the working.

```
int[] temps = {18, 23, 21, 26, 19};
int hotDays = 0;
int total = 0;
foreach (int t in temps) {
    total += t;
    if (t >= 22) hotDays++;
}
Console.WriteLine($"{hotDays}, {total}");
```

The readings 23 and 26 meet the threshold; hotDays is 2. The total is 18 + 23 + 21 + 26 + 19 = 107.

How to answer an unfamiliar coding question

- Name the input, expected output and any constraints.
- Create a state table with one row per iteration.
- Highlight the exact condition that controls each branch.
- Check boundaries: empty input, first/last element and inclusive inequalities.
- Give the precise output before adding your explanation.

Common mistake

A loop variable can change independently of an accumulator. Record both rather than treating each printed value as the value of the loop counter.

2.2.1 · EXAM PRACTICE

SIS: trace and construct

Complete the questions without looking at the marking points.

Q1 · 5 marks Trace the value of score after each iteration and state the final output.

```
int score = 1;
for (int i = 1; i <= 4; i++) {
    if (i % 2 == 0) score += i;
    else score *= 2;
}
Console.WriteLine(score);
```

Q2 · 6 marks An age-checking method should accept ages from 18 to 65 inclusive. Identify the error, write a corrected condition and give four boundary tests (with expected results).

```
static bool ValidAge(int age) {
    return age < 18 || age > 65;
}
```

2.2.1 · EXAM PRACTICE

Writing algorithms in C#

Include a method signature, required initialisation, a valid loop and an explicit result.

Q3 · 8 marks Write static int SumEven(int[] values) to return the sum of even numbers in a non-null array. For {2,3,4}, the result must be 6. Do not modify the array.

Q4 · 6 marks Trace count, then state the time and auxiliary-space complexities if the upper limit 3 is generalised to n.

```
int count = 0;
for (int row = 1; row <= 3; row++)
    for (int col = 1; col <= row; col++)
        count++;
```

2.2.1 · EXAM PRACTICE

Selection with validation

Invalid values must be excluded before their result is considered.

Q5 · 8 marks Write static int CountPasses(int[] scores). Only scores from 0 to 100 inclusive are valid; count those valid scores that are at least 50. Input {49,50,70,-1,101} should return 2. Assume scores is non-null.

2.2.1 · CHECK AND IMPROVE

SIS: marking points

Compare with the mark scheme only after you have attempted all five questions.

Q1 · 5 marks

- score after i=1 is 2 [1]; after i=2 is 4 [1].
- After i=3 it is 8 [1]; after i=4 it is 12 [1].
- Printed output 12 [1].

Q2 · 6 marks

- The original condition returns true for values outside the allowed range [2].
- Correct condition: age >= 18 && age <= 65 [2].
- Boundary tests: 17 false and 18 true [1]; 65 true and 66 false [1].

Q3 · 8 marks

- Signature [1], accumulator initialisation [1], loop through array [2].
- Even-value test using remainder [2], add selected value [1], return sum [1].

```
static int SumEven(int[] values) {
    int total = 0;
    foreach (int v in values)
        if (v % 2 == 0) total += v;
    return total;
}
```

Q4 · 6 marks

- Inner-loop repetitions are 1, 2 and 3 [2]; therefore count = 6 [1].
- For variable n, total iterations 1+2+...+n = n(n+1)/2, hence $O(n^2)$ [2].
- Only loop counters and count are stored, so auxiliary space O(1) [1].

2.2.1 · CHECK AND IMPROVE

SIS: marking points continued

Equivalent valid C# solutions earn credit; method names and variable names may differ.

Q5 · 8 marks

- Method signature returning int [1] and accumulator initialisation [1].
- Correct iteration over scores [2].
- Reject values below 0 or above 100 [2].
- Increment for valid scores at least 50 [1], return the count [1].

```
static int CountPasses(int[] scores) {
    int count = 0;
    foreach (int n in scores) {
        if (n < 0 || n > 100) continue;
        if (n >= 50) count++;
    }
    return count;
}
```

Memory bank · Code First

SIS: sequence / iteration / selection. TRACE: track the state, respect the condition, advance one step, check the output, explain boundaries. TRACE is a memory cue, not OCR terminology.

2.2.1 Programming with purpose

Remember SIS: Sequence, Iteration and Selection. OCR also calls selection branching. Students must show the constructs working together, not just recall the acronym.

SIS appears in a complete method

S SEQUENCE	I ITERATION	S SELECTION
Read calculate	Repeat for items	Choose by condition

Precise exam wording: sequence, iteration and branching.

NOW BE PRECISE

Sequence executes instructions in order. Selection chooses a branch using a Boolean condition. Iteration repeats instructions. Modularity breaks a solution into reusable subprograms, and scope controls where identifiers can be accessed.

WORKED EXAMPLE 1 / Score tracker

A method processes marks, rejects invalid values outside 0–100, and counts valid marks at or above 50. It needs an array (input), iteration through marks, selection for validation/pass conditions and a returned count (output).

C# / model method

```
static int PassCount(int[] marks) {
    int count = 0;
    foreach (int m in marks) {
        if (m >= 0 && m <= 100 && m >= 50)
            count++;
    }
    return count;
}
```

For [45, 50, 107, 72], valid passing marks are 50 and 72, so the result is 2. The loop iterates once per element; the IF is selection; initialization and return are part of sequence.

WORKED EXAMPLE 2 / Different method

Write the steps for a method that counts valid failing marks (0 to 49) in [14, 56, -2, 0, 99].

Guided response

Loop through the array, test 0 <= mark < 50, increment a counter for matches and return the counter. The result is 2 (14 and 0); -2 is invalid.

State: procedural return vs object method

Methods, state and the meaning of a return value

PROCEDURAL	OBJECT METHOD
balance = Deposit(8,4)	account.Deposit(4)
result = 12	object state = 12

An ignored returned value does not update the caller variable.

A procedural method may return a new value without changing the caller variable. An object method may deliberately update the state of one object. Keep the distinction clear when tracing C#.

WORKED EXAMPLE 1 / Return ignored

```
static int Deposit(int balance, int amount) {
    return balance + amount;
}
int balance = 8;
Deposit(balance, 4);
Console.WriteLine(balance); // 8
```

The method returns 12, but the caller does not assign it. The caller remains 8. To update the caller explicitly, use balance = Deposit(balance, 4);

WORKED EXAMPLE 2 / Separate objects

Account a starts with 8; Account b starts with 20. Calling a.Deposit(4) changes the first object only, if Deposit updates that object's balance. What are both balances afterwards?

Guided response

a becomes 12; b stays 20. They are independent object instances. The exact behaviour still depends on the method implementation.

IMPORTANT / RETURN VALUES AND STATE

Both procedural and object methods may return values or mutate state. Trace the caller variable and each object field separately.

COMMON MISTAKE

Do not assume that all calls mutate variables. C# passes ordinary int arguments by value unless ref/out is used; a method can also return a value for the caller to assign.

Exam Practice

Q1 · 4 marks Identify one example of sequence, iteration and selection in the PassCount method. State the result of PassCount([49,50,75,101]).

Q2 · 6 marks Write a C# method CountValid that returns how many values in an integer array are between 0 and 100 inclusive.

Q3 · 4 marks Trace the values of a, b and c after: int a=5; int b=a; b=b+3; int c=a+b;

Q4 · 6 marks Compare the effect of calling Increase(int n) and Increase(ref int n), where each increments n by 2 and the caller starts at x=7.

Q5 · 6 marks A booking system has methods Validate, CheckSeats, SaveBooking and Confirm. Explain how modularity helps and why SaveBooking must succeed before Confirm runs.

Model answers and marking points

Attempt the questions before checking your responses. Other technically accurate explanations may also earn credit.

Q1

Sequence: initialising count or returning it [1]; iteration: foreach [1]; selection: if condition [1]. Result for [49,50,75,101] is 2 (50 and 75 only) [1].

Q2

Example: static int CountValid(int[] a) { int count=0; foreach (int x in a) { if (x>=0 && x<=100) count++; } return count; } Mark inputs/return [1], counter initialisation [1], loop [1], correct inclusive condition [2], increment/return [1].

Q3

a remains 5 [1], b becomes 8 [1], c=5+8=13 [1], and the assignment b=a made a copy of the integer value, not a link to a [1].

Q4

Without ref, the parameter receives a value copy [1]; its local value becomes 9 [1], but the caller x remains 7 [1]. With ref the parameter aliases x [1], the increment changes caller x to 9 [1], and changes persist after the call [1].

Q5

Small methods isolate distinct operations for testing [1], reuse [1] and clearer fault location [1]. CheckSeats should precede saving [1], confirmation must wait for a successful committed save [1], otherwise users could be told seats were reserved when they were not [1].

Theory / Trace a recursive function

Trace the short C# function below. Follow each recursive call and return while keeping the code visible.

```
int Factorial(int n)
{
    if (n == 0) return 1;
    return n * Factorial(n - 1);
}
```

Look for	What it tells us
Base case	When n is 0, return 1 without making another call.
Recursive case	Otherwise call Factorial(n - 1).
Progress	The argument moves towards 0 on each call.
Return	Each earlier call waits for the value from the next call.

Key vocabulary

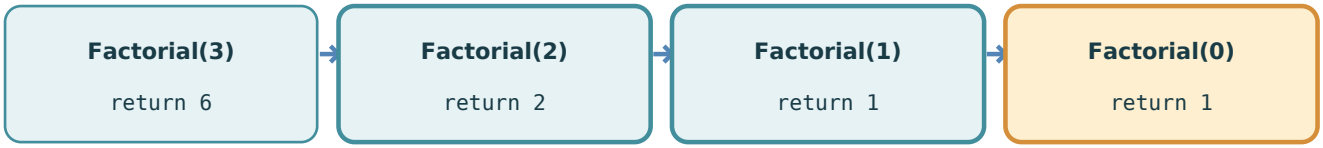
Keyword	Exam-accurate meaning
Recursion	A routine invoking itself to solve a smaller instance of a problem.
Base case	A condition that returns without making another recursive call.
Call stack	Runtime storage of active function calls, their local state and return points.
Stack frame	The information retained for one active function call.
Unwinding	Returning from the deepest call to the earlier suspended calls.

Prior knowledge: functions return a value; a C# `return` statement ends that invocation; local variables belong to their own function call.

02 / WORKED EXAMPLE 1 - follow all the calls

Task: trace `Factorial(3)` fully. The recursive call is part of the multiplication, so the first call cannot finish until later calls have returned.

Calls go down; return values resolve upwards



CALLS →

RETURNS ← Each suspended call resumes after the inner call returns.

Stage	Reasoning
Calls	Factorial(3) → Factorial(2) → Factorial(1) → Factorial(0)
Base case	Factorial(0) returns 1; there are no more calls.
Return 1	Factorial(1) = 1 × 1 = 1.
Return 2	Factorial(2) = 2 × 1 = 2.
Return 3	Factorial(3) = 3 × 2 = 6.

Call stack: why return order matters

Each call has its own value of n. A suspended frame retains the unfinished multiplication. The stack is last-in, first-out: the call with n=0 returns first, and the call with n=3 finishes last.

Common misconception • Recursion is not automatically an infinite loop. It terminates when a reachable base case stops further calls. Missing the base case, or failing to move towards it, can exhaust the call stack.

03 / WORKED EXAMPLE 2 - new input

Work without running the program. Use the same factorial function from the previous page, but now trace Factorial(4).

STEP 1 / FILL THE CALLS

Call position	Function call	Value returned (fill backwards)
1	Factorial(4)	_____
2	Factorial(__)	_____
3	Factorial(__)	_____
4	Factorial(__)	_____
5	Factorial(__)	_____

STEP 2 / IDENTIFY THE BASE CASE

The base case occurs when n = _____ . Its returned value is _____ .

STEP 3 / UNWIND THE RETURNS

Factorial(1) = _____

Factorial(2) = _____

Factorial(3) = _____

Factorial(4) = _____

Guided check

Calls: 4 → 3 → 2 → 1 → 0. The base case returns 1. Then the frames return 1, 2, 6 and 24 in that order.

Explain: Why can’t Factorial(4) return 4 immediately when it first runs?

_____

_____

Recursion — code for independent practice

Keep these two C# functions visible when answering the questions on the next page.

Function A · Factorial

```
int Factorial(int n)
{
    if (n == 0) return 1;
    return n * Factorial(n - 1);
}
```

Assume n is non-negative for the original definition. The base case is reached at n = 0.

Function B · SumTo

```
int SumTo(int n)
{
    if (n == 0) return 0;
    return n + SumTo(n - 1);
}
```

How to write a recursive trace

- First list the argument used by every function call.
- Identify the point where the base case returns without further calls.
- Write returned values from the deepest call back to the first.
- For a negative input, ask whether each recursive argument moves towards the base case.

Try the three independent questions on the next page. The marking guidance follows later in the chapter.

Recursion · independent call-stack practice

Q1 [3 marks] Write the calls and returned values for Factorial(2).

WORKING / METHOD

Q2 [4 marks] Trace SumTo(3) for the function shown on the preceding page, including base case, calls, return sequence and final result.

WORKING / METHOD

Q3 [3 marks] Explain what goes wrong with Factorial(−1), then write a valid C# guard to reject negative arguments.

WORKING / METHOD

Recursion — SumDigits code reference

Use this function for the multipart independent question on the following pages.

Original C# algorithm

```
int SumDigits(int n)
{
    if (n < 10) return n;
    return n % 10 + SumDigits(n / 10);
}
```

What the code means

- Input assumption: n is a non-negative integer.
- Integer division n / 10 removes the last decimal digit.
- n % 10 gives the final decimal digit of n.
- Each function call waits for the inner call to return.
- The n < 10 test stops recursion once only one digit remains.

Trace structure — without the answer

Original input › **Smaller input** › **Base case** › **Unwind returns**

Student task: complete the call and return sequences independently. Do not confuse the two directions.

Recursion: SumDigits application (see code on previous page)

Q1 [2 marks] (a) State the base case of SumDigits and explain why recursion stops.

Q2 [3 marks] (b) Trace the function calls for SumDigits(274).

WORKING / METHOD

Q3 [3 marks] (c) Calculate the final returned result, showing the order of returns.

WORKING / METHOD

Recursion: SumDigits application (see code on previous page)

Q4 [4 marks] (d) Explain one advantage and one potential disadvantage of recursion compared with an iterative solution.

06 / Model answers and misconceptions

Independent practice / Questions 1–3

Q	Model answer and allocation
1 [3]	Calls 2 → 1 → 0 [1]. Base case returns 1 [1]. Returns: Factorial(1)=1 and Factorial(2)=2 [1].
2 [4]	Base case n=0 returns 0 [1]. Calls: 3 → 2 → 1 → 0 [1]. Returns: 0, 1, 3, 6 [1]. Final result 6 [1].
3 [3]	For n=-1 the argument decreases further, so it never reaches 0 [1]; the stack can overflow [1]. E.g. `if (n < 0) throw new ArgumentOutOfRangeException(nameof(n));` before the base case [1].

Exam application / Question 4

Part	Model answer and marking points
(a) [2]	When n < 10, return n [1]. No further recursive call is made [1].
(b) [3]	SumDigits(274) → SumDigits(27) → SumDigits(2) [1 each correct call/transition, maximum 3].
(c) [3]	SumDigits(2) returns 2 [1]. SumDigits(27) returns 7+2 = 9 [1]. SumDigits(274) returns 4+9 = 13 [1].
(d) [4]	Recursion may express a self-similar problem clearly [1], for example decomposing a number digit by digit [1]. It uses additional stack frames [1], which can increase memory use or risk stack overflow for deep recursion [1]. Other coherent evaluated comparisons acceptable.

Common mistakes

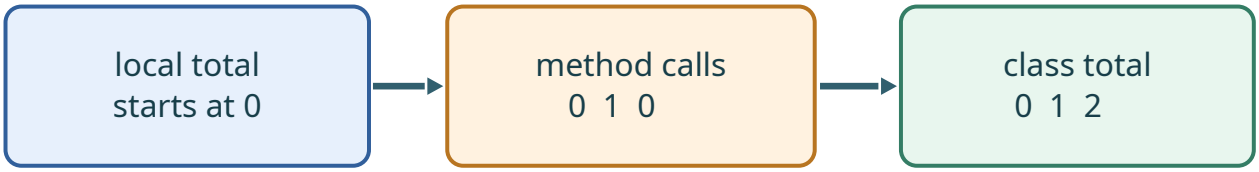
Treating each call as a new loop iteration; returning values in call order instead of reverse order; omitting a reachable base case; forgetting integer division; assuming that using recursion is always faster.

Global and local scope: trace the state

Theory / Worked Examples

Scope determines where a variable name can be accessed; lifetime determines how long its value exists. In C#, local variables and static fields are not interchangeable.

Local state and shared state



The exact outcome depends on which declaration the method updates.

REMEMBER THIS

Your notebook versus the classroom noticeboard. A local variable is a fresh notebook for this call. A class-level static field is a shared noticeboard that can persist between calls.

Now be precise

• **Local:** declared inside a method/block and accessible only within its scope. Each call normally has a distinct local activation.

• **Class-level field:** declared in a class; static fields are associated with the type, while instance fields belong to separate objects. Avoid describing every field as global.

• **Shadowing:** a local variable can have the same name as a field. Use `Counter.total` or `this.total` where appropriate to clarify which storage is read.

WORKED EXAMPLE 1

A static field total starts at 0. A method has local int temp=0; each call increments temp but leaves total unchanged. What is total after two calls?

Method: Both calls create their own temp; total remains 0. If the method instead executes total++, it becomes 2 after two calls.

WORKED EXAMPLE 2

Two instances have Score fields 4 and 9. Only first instance calls Add(3).

Method: First instance field becomes 7; second remains 9. Instance fields are not shared just because their names are identical.

COMMON MISTAKE

A local variable named total does not automatically update a field named total.

Theory / Key terminology

Use these exam-accurate terms when explaining algorithms and programming.

Scope	The region of a program in which an identifier can be used.
Local variable	Variable accessible within its declared subroutine or block.
Global variable	Variable declared outside local routines, available across a wider scope.
Lifetime	The period during execution for which a value or variable exists.
Shadowing	A local identifier hides another identifier with the same name.
Parameter	A named input received by a subroutine.

Exam practice

Write independently. Marks are shown beside each question.

Q1 · 3 marks. Define local variable and class-level field, identifying where each can be used.

Q2 · 4 marks. A method executes int local=0; local++; return local;. It is called twice. State both return values and explain why they do not accumulate.

Q3 · 4 marks. A static field Counter.total begins 0. Increment it twice, while a separate local total begins 5. Give both final values and explain the difference.

Mark scheme

Compare only after completing the questions.

Q1 · 3 marks

Local declaration within method/block [1], accessed within scope [1]; class-level field declared on type and accessible subject to access modifiers [1].

Q2 · 4 marks

Each call returns1 [2]; new local is initialised each call [1]; no persisting field is updated [1].

Q3 · 4 marks

Counter.total=2 [1]; local total=5 if unchanged [1]; separate variables [1]; static field persists across calls while local exists for its own scope [1].

RECALL

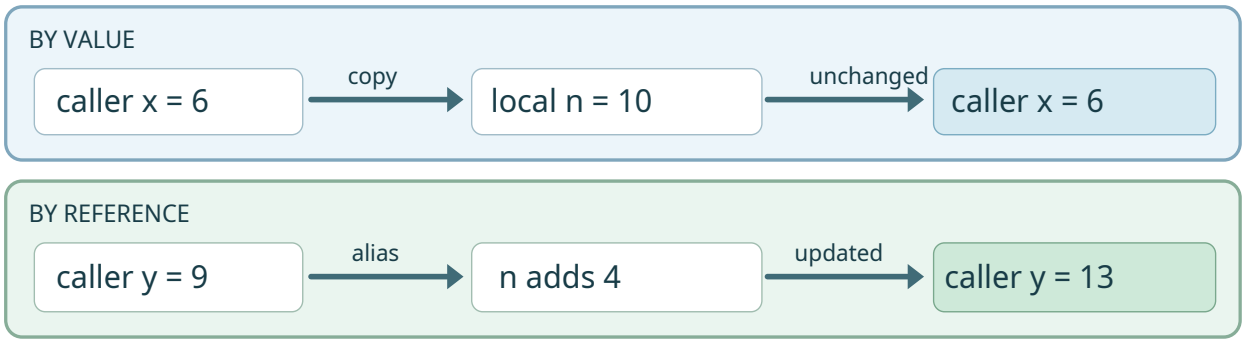
Your notebook versus the classroom noticeboard: A local variable is a fresh notebook for this call. A class-level static field is a shared noticeboard that can persist between calls.

2.2.1 PROGRAMMING TECHNIQUES / Parameters, file handling and modular coding

REMEMBER / PHOTOCOPY OR ORIGINAL

Passing by value is like working on a copy of a number. Passing by reference is like giving a method access to the caller's actual variable. In C#, `ref` permits a method to update that caller variable.

Passing parameters: copy versus alias



WORKED EXAMPLE 1 / Passing by value

C# TRACE

```
static void AddFour(int n) { n += 4; }
int x = 6;
AddFour(x);
```

Within the method, n becomes 10; the caller's x remains 6.

WORKED EXAMPLE 2 / new input

Change the method parameter to `ref int n`, call it with `ref y`, and begin with `y = 9`. What does the caller contain afterwards?

GUIDED CHECK

Because n aliases the caller's y, adding four changes y from 9 to 13. A by-value argument would leave y unchanged.

EXAM PRECISION / C# PARAMETER PASSING

An object reference passed by value can still allow changes to object fields; it does not let a method reassign the caller variable. The ref keyword allows that reassignment.

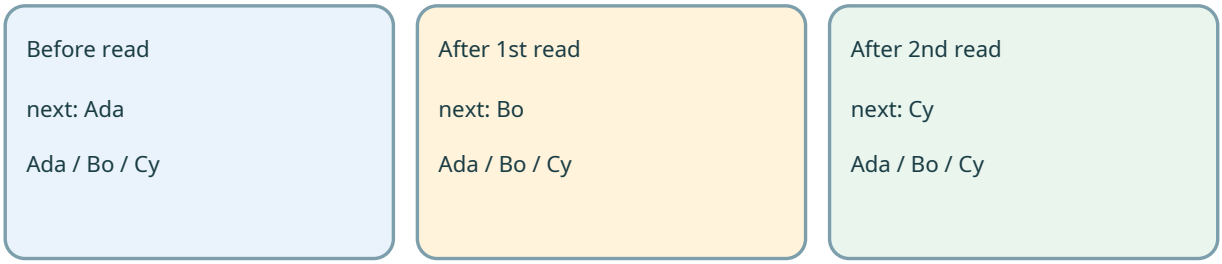
Theory / Key terminology

Use these exam-accurate terms when explaining algorithms and programming.

Procedure	Named subroutine that performs a task without returning a function value.
Function	Subroutine that returns a value to its caller.
Formal parameter	Parameter named in a subroutine's definition.
Actual argument	Value or variable supplied when a subroutine is called.
Pass by reference	Passing access to a variable so a subroutine can modify the caller's value.
Modularity	Organising a program into separate, purposeful components.
File handling	Opening, reading, writing and closing persistent files.

OCR exam reference language / files

Reading a text file moves a position forward



openRead does not overwrite data; always close the file.

WORKED EXAMPLE 1 / Reading a text file

OCR EXAM REFERENCE

```
f = openRead("names.txt")
while NOT f.endOfFile()
    print(f.readLine())
endwhile
f.close()
```

Each readLine retrieves the next line. The while condition stops the loop at EOF.

WORKED EXAMPLE 2 / different file

The lines in `cities.txt` are York then Bath. How many times does `readLine()` run, and what is printed?

GUIDED CHECK

Two reads; print York, then Bath. The file pointer advances each time. The file must be closed when finished.

EXAM PRECISION

Use OCR notation when the task requests it. Distinguish openRead (read existing data) from openWrite (writing can replace existing data). In C#, always mark actual ref parameters and calls explicitly.

Parameters, file handling and modular coding / exam practice

Q1 [4 marks] A C# method AddFour(int n) adds 4 to its parameter. It is called with x = 6. State the parameter value inside the method and the caller value afterwards; explain why they differ.

Q2 [6 marks] Write a C# procedure that adds 3 to a caller integer using ref, and show a call where the variable starts at 5. State its value afterwards.

CODE / PSEUDOCODE

Parameters, file handling and modular coding / exam practice

Q3 [6 marks] Write OCR A Level exam reference pseudocode to open names.txt, print all lines until end of file and close it.

CODE / PSEUDOCODE

Q4 [5 marks] Compare passing by value with passing by reference, including one advantage of returning a result instead of changing caller state directly.

Check your answers — parameters and files

Use equivalent valid C# or OCR examination reference language as the question requires.

Q1 · 4 marks · Passing by value

- The local parameter n initially receives 6. [1]
- Inside AddFour, n becomes 10. [1]
- The caller variable x remains 6. [1]
- The caller is unchanged because int is passed by value. [1]

Q2 · 6 marks · Passing by reference

- Declare a void method that takes ref int. [1]
- Change the parameter by adding 3. [1]
- Use valid C# syntax in the method. [1]
- Create int x = 5 in the caller. [1]
- Pass ref x at the call site. [1]
- After the call, x is 8. [1]

```
static void AddThree(ref int n) { n += 3; }
int x = 5;
AddThree(ref x);  // x = 8
```

Q3 · 6 marks · OCR ERL file handling

- Open names.txt using openRead. [1]
- Set up a loop that stops at endOfFile(). [1]
- Read a complete line on each iteration. [1]
- Print the line read. [1]
- Continue correctly until the end of the file. [1]
- Close the opened file. [1]

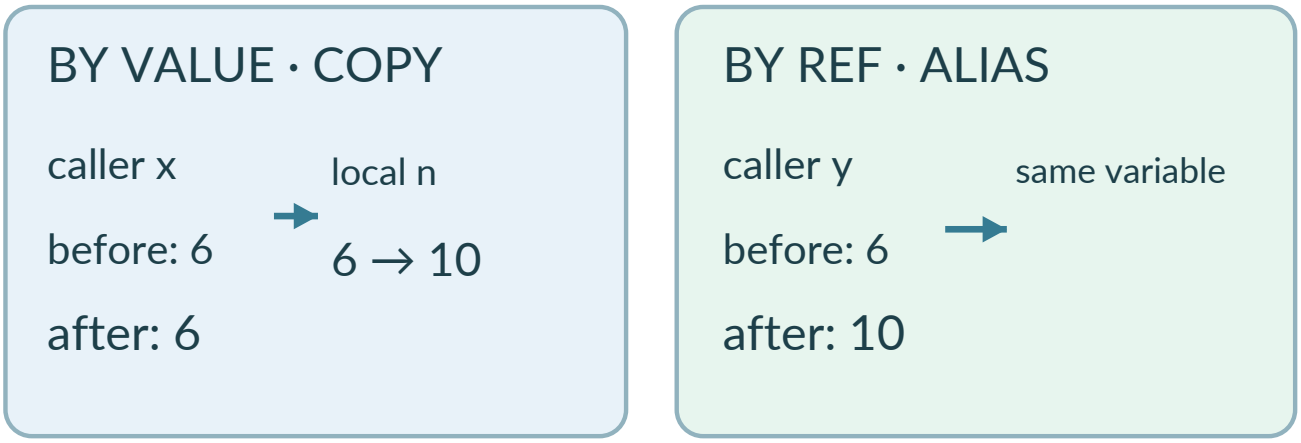
```
f = openRead("names.txt")
while NOT f.endOfFile()
    print(f.readLine())
endwhile
f.close()
```

Q4 · 5 marks · Compare approaches

- By value passes a copy. [1]
- Modifying a by-value int parameter does not alter the caller variable. [1]
- By reference can modify the caller variable. [1]
- Returning a result makes the change explicit at the call site. [1]
- Explicit returned results can make a method easier to test or reason about. [1]

Parameters, copies and shared state

One variable can be protected from changes or deliberately passed for modification. Understanding exactly which state changes earns the marks.



WORKED EXAMPLE 1 · Value versus ref

```
static void CopyAdd(int n) { n += 4; }
static void RefAdd(ref int n) { n += 4; }
int x = 6, y = 6;
CopyAdd(x);
RefAdd(ref y);
Console.WriteLine($"{x}, {y}");
```

The output is 6, 10. CopyAdd changes a local copy. RefAdd aliases the caller variable because both declaration and call use ref.

Remember this

Photocopy versus original: drawing on a photocopy does not change the original; editing the original document does. Limit: C# arrays are reference types, and a copied reference can still access the same array elements.

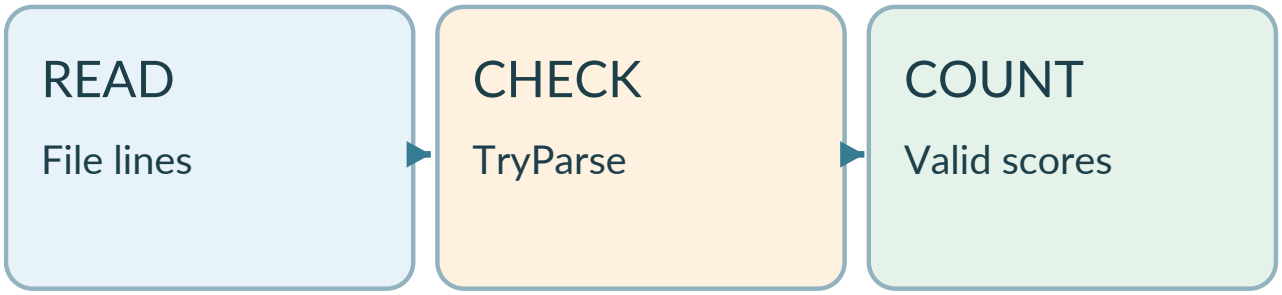
WORKED EXAMPLE 2: the array reference trap

Passing an array without ref copies the reference, not the array elements. Predict the final caller value.

```
static void Change(int[] a) {
    a[0] = 9;
    a = new int[] {1, 2};
}
int[] numbers = {4, 5};
Change(numbers);
Console.WriteLine(numbers[0]);
```

Output: 9. Both caller and parameter initially refer to the same array, so changing element 0 is visible. Reassigning the local parameter a to a new array does not rebind the caller variable numbers.

File handling: data enters the programme




```
using System;
using System.IO;
string[] lines = File.ReadAllLines("scores.txt");
int count = 0;
foreach (string line in lines) {
    if (int.TryParse(line, out int value) &&
        value >= 0 && value <= 100) count++;
}
Console.WriteLine(count);
```

Assume the file exists and is readable. TryParse safely rejects non-integer lines; it does not throw a FormatException for text such as bad.

2.2.1 · EXAM PRACTICE

Parameters and caller state

Show the state visible to the caller, not just the value inside a method.

Q1 · 5 marks Predict the printed values and explain the different effects of these calls.

```
static void DoubleCopy(int n) { n *= 2; }
static void ReduceRef(ref int n) { n -= 3; }
int a = 8, b = 8;
DoubleCopy(a); ReduceRef(ref b);
Console.WriteLine($"{a}, {b}");
```

Q2 · 6 marks Predict the output, distinguish element mutation from parameter reassignment, and explain why the caller variable still refers to its original array.

```
static void Change(int[] x) {
    x[0] = 9;
    x = new int[] {1, 2};
}
int[] a = {4, 5};
Change(a);
Console.WriteLine(a[0]);
```

2.2.1 · EXAM PRACTICE

File algorithms

Assume any file named in a question exists and can be read. Use C# and state any additional assumptions.

Q3 · 8 marks Write static int CountNonBlankLines(string path). Read the lines of the specified text file; count only lines that are not empty or whitespace; return the count. Use File.ReadAllLines and string.IsNullOrWhiteSpace.

Q4 · 6 marks The file contains three lines: 20, bad and 30. Explain why the following method fails, then correct the parsing and describe the expected valid-number total.

```
int total = 0;
foreach (string line in File.ReadAllLines("scores.txt")) {
    int score = int.Parse(line);
    total += score;
}
Console.WriteLine(total);
```

2.2.1 · EXAM PRACTICE

A method with a contract

A useful method makes its preconditions and side effects explicit.

Q5 · 6 marks Write a C# method Reserve(ref int available, int requested) returning bool. It returns false without changing available when requested is not positive or is greater than available. Otherwise subtract requested from available and return true.

Exam precision

Mention precondition when it matters: here available should start at zero or above. The question states what must happen for invalid requested values.

2.2.1 · CHECK AND IMPROVE

Parameters and files: marking points

Award marks only for steps that the question requires. Equivalent correct C# is acceptable.

Q1 · 5 marks

- a remains 8 [1]; b becomes 5 [1].
- DoubleCopy modifies only its local parameter [1].
- ReduceRef modifies the caller variable [1]; ref is present at both declaration and call [1].

Q2 · 6 marks

- The array element becomes 9 [1]; the output is therefore 9 [1].
- The array reference is passed by value [2]; both references initially point to the same array.
- Reassigning parameter x changes only that local reference, not caller a [2].

Q3 · 8 marks

- int return type and valid method signature [1]; read lines using File.ReadAllLines(path) [1].
- Iterate across returned lines [2]; reject null/empty/whitespace with IsNullOrWhiteSpace [2].
- Increment a count [1]; return the count [1].

```
static int CountNonBlankLines(string path) {
    int count = 0;
    foreach (string line in File.ReadAllLines(path)) {
        if (!string.IsNullOrWhiteSpace(line)) count++;
    }
    return count;
}
```

2.2.1 · CHECK AND IMPROVE

Parameters and files: continued

Recall the difference between parsing an integer and validating its permitted range.

Q4 · 6 marks

- int.Parse("bad") causes a FormatException [2].
- Use int.TryParse to skip invalid lines [2].
- 20 and 30 are valid integers and total 50 [2].

```
int total = 0;
foreach (string line in File.ReadAllLines("scores.txt")) {
    if (int.TryParse(line, out int score))
        total += score;
}
Console.WriteLine(total); // 50
```

Q5 · 6 marks

- Boolean method signature [1], correct ref parameter [1].
- Guard against requested <= 0 and requested > available [2].
- Subtract from available on success [1]; return true/false as specified [1].

```
static bool Reserve(ref int available, int requested) {
    if (requested <= 0 || requested > available)
        return false;
    available -= requested;
    return true;
}
```

Memory bank

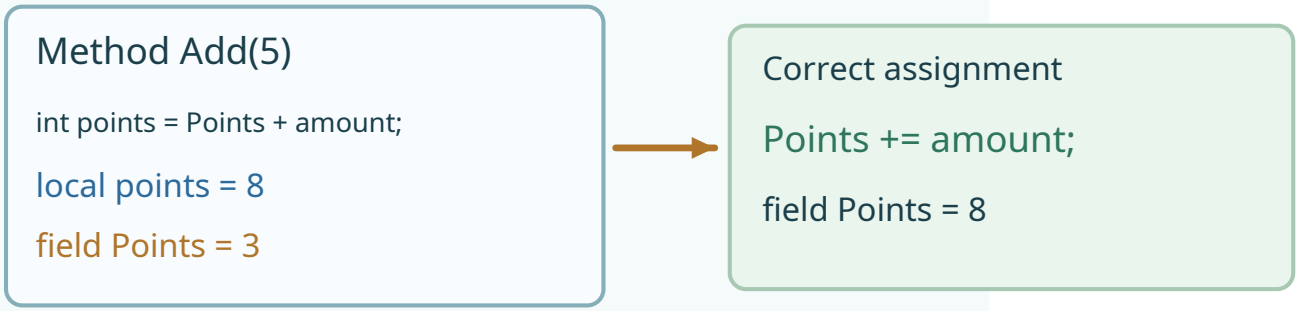
By value: caller variable protected from parameter assignment. By ref: caller variable may be updated. Array passed by value: the copied reference can still mutate shared elements. TryParse: guards against invalid text input.

2.2.1 / Debugging with an IDE

Breakpoints, stepping, watching variables and repair

REMEMBER THIS A debugger is a detective: stop at the scene, inspect evidence, follow one step, retest.

DEBUG THE STATE, NOT JUST THE OUTPUT



Step Into Watch local and field Fix Run tests again

NOW BE PRECISE Do not confuse Step Over (executes a call as one step) with Step Into (moves inside the called method).

KNOW IT PRECISELY

Breakpoint. Pauses execution at a selected line so variable state can be inspected.

Step Into. Enters a called function or method.

Step Over. Executes a function call without displaying its internal steps.

Watch. Displays the value of selected variables or expressions while paused.

Call stack. The active sequence of calls leading to the current execution location.

Theory / Key terminology

Use these exam-accurate terms when explaining algorithms and programming.

Breakpoint	A location where a debugger pauses execution for inspection.
Step into	Run a statement and enter a called subroutine.
Step over	Run a statement without stepping through a called subroutine.
Watch window	IDE display that tracks expressions or variable values during execution.
Syntax error	Code that violates the programming language's grammatical rules.
Logic error	Code runs but produces an incorrect result.
Runtime error	Failure encountered while a program is executing.

Debugging with an IDE / Worked method

Worked examples and a different guided problem.

UNDERSTAND IT

- A Score object has Points=3. Calling Add(5) mistakenly updates a local points variable to 8, leaving the object field Points=3.
- Place a breakpoint on the call, Step Into Add, watch both points and Points, then replace the faulty assignment with Points += amount.
- A separate array error: for (int i=0; i<=a.Length; i++) accesses a[a.Length], which is outside valid indexes. Replace <= with <.

WORKED EXAMPLE 1 / INSPECT AN OBJECT

Watch Points before Add(5): 3. Step Into Add. After the erroneous local assignment: points=8, Points=3. Correct the field assignment, rerun and verify Points=8.

CHECK WITH A TEST

Given Points initially 3, Add(5) should produce 8; Add(0) should preserve 3. Include these assertions after the repair.

WORKED EXAMPLE 2 / FIND OFF-BY-ONE

Array [4,7,9] has Length=3 and valid indexes 0–2. A loop using i<=Length eventually reaches i=3. Break on the indexing line, inspect i and Length, and correct it to i<Length.

Debugging a complete C# method

Trace the mechanism carefully; these examples are not questions to memorise.

WORKING EXAMPLE FROM THE SCORE OBJECT

C# / OCR-style code

```
class Score {
    public int Points { get; private set; } = 3;
    public void Add(int amount) {
        int points = Points + amount; // BUG
    }
}

Score score = new();
score.Add(5);
Console.WriteLine(score.Points);
```

Debugger moment	Watch values
Before Add(5)	score.Points = 3
After local assignment	points = 8, score.Points = 3
After correction	score.Points = 8

FIX Replace the assignment with Points += amount; then rerun the test and inspect the resulting object state.

Debugging with an IDE / Exam practice

Complete the questions independently. Use the space provided for your working.

Q1 4 MARKS

A method Add(5) changes local variable points from 3 to 8, but its object field Points remains 3. Describe how to use a breakpoint and Watch to find the fault.

Q2 5 MARKS

Correct the faulty C# loop `for (int i=0; i<=a.Length; i++) Console.WriteLine(a[i]);` and explain the error for an array of length 3.

Debugging with an IDE / Exam practice

Complete the questions independently. Use the space provided for your working.

Q3 6 MARKS

Explain how Step Into, Step Over, the call stack and repeat testing help debug a nested method call.

Debugging with an IDE / Check your answers

Use these mark points after completing the independent questions.

Q1 / 4 MARKS

[1] Breakpoint pauses before or within Add.

[2] Step Into Add and inspect the assignment.

[3] Watch local points: it becomes 8.

[4] Watch field Points: it remains 3, exposing the wrong assignment target.

Q2 / 5 MARKS

[1] Use i < a.Length, not <=.

[2] Valid indexes for length3 are 0,1,2.

[3] The original loop reaches i=3.

[4] a[3] causes an index-out-of-range fault.

[5] Re-run with a 3-element array to confirm only 4,7,9 are printed.

Q3 / 6 MARKS

[1] Step Into reveals operations inside the called method.

[2] Step Over executes the call as one step in the caller.

[3] Call stack shows the nested chain of active calls.

[4] Watching arguments and locals reveals where state diverges from expectations.

[5] Re-run after a correction to confirm the intended result.

[6] Use regression/boundary tests to ensure the change does not create other faults.

Debugging objects and methods

Use a variable watch to prove which value has changed, rather than guessing.

REMEMBER THIS / TWO NAME TAGS

A local variable called **points** and an object field called **Points** are different locations. Same-sounding names do not make them aliases. Watch **both** during debugging.

WATCH THE FIELD, NOT THE LOCAL VARIABLE

BEFORE	INSIDE Add(5)	AFTER BUG
Points = 3	local points = 8	Points still = 3

Wrong: int points = Points + amount;

Fix: Points += amount;

Step Into / Locals: points 8 / Watch: Points 3

```
class Score {
  public int Points { get; private set; } = 3;
  public void Add(int amount) {
    int points = Points + amount; // BUG
  }
}
var score = new Score();
score.Add(5);
Console.WriteLine(score.Points); // 3, not 8
```

WORKED EXAMPLE 1: Step Into Add(5). Locals displays local points=8, while Watch(score.Points) remains 3. The method changes only a temporary local variable. Replace the faulty assignment with **Points += amount;**. The next run prints 8.

WHAT THE IDE CAN REVEAL

Breakpoint pauses execution; **Step Into** enters a method; **Step Over** runs a method without following its statements; **Locals** displays in-scope local state; **Watch** monitors a chosen expression.

Theory / Key terminology

Use these exam-accurate terms when explaining algorithms and programming.

Class	Blueprint defining attributes and methods for a kind of object.
Object	An instance of a class with its own state.
Constructor	Special routine used to initialise a new object.
Encapsulation	Combining data and methods and controlling access to state.
Inheritance	Deriving a class from another to reuse and extend behaviour.
Polymorphism	Using a shared interface with behaviour that depends on the object.
Method overriding	Providing a derived-class implementation of an inherited method.

Trace overridden methods: new case

```
class Message {
  public virtual string Text() => "Ready";
}
class Warning : Message {
  public override string Text() => "Check";
}
Message m = new Warning();
Console.WriteLine(m.Text());
```

WORKED EXAMPLE 2 / STEP THE CALL

Predict which method runs and the printed word. Explain why the declared type of m does not force the base Text() method to run. Then modify the example to create **new Message()** instead and predict the new output.

Check: First prints Check through the subclass override; second prints Ready because the object really is Message. This is polymorphism. In both cases Text() returns a string and does not modify state.

EXAM PRECISION

An access modifier controls which code may access a member. The runtime object type determines overridden virtual method behaviour. Not every same-named method is a valid override.

Exam practice: OOP programming

Q1 [5 marks] In the Score code, identify the bug, predict output and state how to correct it. Name one IDE feature to confirm the diagnosis.

Q2 [6 marks] Write a C# class Counter with a private integer field, a constructor setting its initial value, an Increase() method and a public way to read the current count.

Exam practice: OOP programming

Q3 [6 marks] Explain what Message m = new Warning(); does and why m.Text() returns Check. Give the result for Message m = new Message();

Q4 [4 marks] A class has public int Balance and a second version has private int balance plus validated Deposit(). Explain one risk of the first and one benefit of the second.

Check your answers: OOP programming

Award each mark for a distinct, technically correct point. Equivalent valid C# is acceptable.

Q1 · 5 marks

1. The method creates or updates a **local variable** called points rather than assigning to the Points property.
2. Therefore the object's Points value is unchanged after Add(5).
3. The program outputs **3**.
4. Correct the body to **Points += amount;** (or an equivalent assignment to the property).
5. Use Step Into with Watch/Locals or a breakpoint to compare the local and object values.

Q2 · 6 marks

1. Declare a C# class named Counter.
2. Declare a private integer field to hold the count.
3. Provide a constructor accepting an initial integer value.
4. Assign the constructor argument to that field.
5. Provide a public Increase() method that increments the field.
6. Provide a public getter/property that returns the current value; use valid, consistent C# syntax.

Q3 · 6 marks

1. m is declared with the base type Message.
2. new Warning() creates a Warning object, which m references.
3. Warning inherits from Message.
4. Warning overrides the virtual Text() method.
5. Runtime dispatch prints **Check** for the Warning instance.
6. A new Message() instance prints **Ready**.

Q4 · 4 marks

1. A public writable Balance field may be assigned directly.
2. This can bypass checks and allow an invalid balance or violate an invariant.
3. A private field prevents external code from assigning the value directly.
4. A validated Deposit() method can reject invalid changes before updating the field.

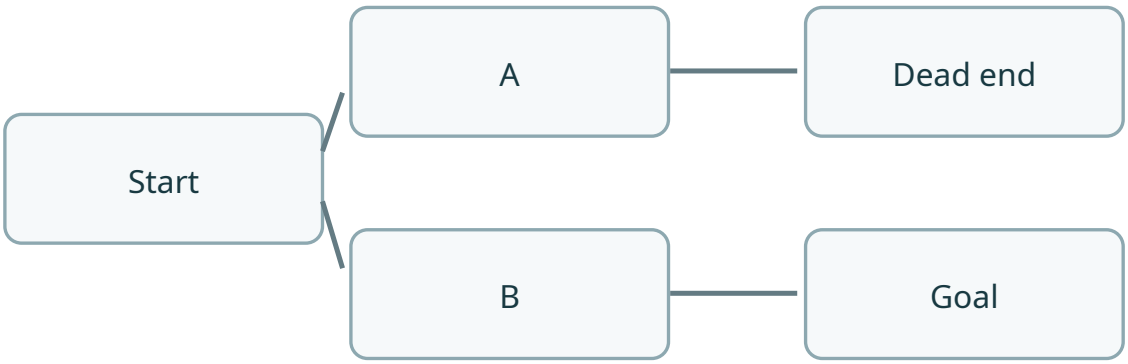
Example C# implementation

```
class Counter {
private int count;
public Counter(int start) { count = start; }
public void Increase() { count++; }
public int Count => count;
}
```

2.2.2 / Computational methods

Remember: TRY → TEST → UNDO → RETRY: backtracking.

BACKTRACKING: TRY, TEST, UNDO, RETRY



A fails; undo that choice and explore B to reach Goal.

ANALOGY - PICTURE IT

Exploring a maze: take a route, reach a dead end, return to the last junction, then try another route.

NOW BE PRECISE - WHERE THE ANALOGY STOPS

Backtracking searches a structured choice space; efficient pruning can help, but worst-case work may still be exponential.

THEORY / KEY TERMINOLOGY

Decomposition	Divide a problem into smaller subproblems.
Divide and conquer	Solve smaller subproblems and combine their solutions.
Backtracking	Build candidate solutions, undo unpromising choices and try alternatives.
Heuristic	Useful rule of thumb that guides search but may not guarantee an optimal result.
Performance modelling	Use a simplified model to estimate behaviour or resource demand.
Pipelining	Overlap stages of work by processing several items at different stages.

Computational methods / understand and model

A problem suitable for computational methods has definable inputs, operations and criteria for recognising an acceptable solution. Abstraction excludes irrelevant detail; decomposition breaks the task into manageable units. Data mining identifies patterns in large data; visualisation makes results interpretable. Backtracking explores and undoes choices, whereas heuristics prioritise promising possibilities. Performance modelling predicts system behaviour; pipelining can increase throughput by overlapping independent stages, though it may not reduce an individual item's latency.

WORKED EXAMPLE 1 / worked example

Problem: find a route from Start to Goal. At Start, routes A and B are candidates.

Try: select A and extend it. A leads to a dead end.

Undo: return to Start, remove A from the current candidate route, then explore B.

Success: B reaches Goal; record the valid path.

WORKED EXAMPLE 2 / same skill, different problem

YOUR TURN

A timetable solver assigns Room 1 to a class but discovers a clash with a later lesson. Describe a backtracking solution and one strategy for trying promising rooms first.

Scaffold: Describe the point where the solver detects an invalid partial assignment and reverses it.

My working:

GUIDED CHECK - READ AFTER ATTEMPTING

Try: Place the class in a candidate room.
Undo: Remove the choice after a clash.
Heuristic: Try the room with fewest competing classes first.

Computational methods · independent exam practice

Q1 [4 marks] Distinguish abstraction and decomposition in a sports app.

DIAGRAM / EXPLANATION

Q2 [4 marks] Compare divide and conquer with backtracking.

Q3 [4 marks] Give one advantage and one limitation of a heuristic.

Computational methods · independent exam practice

Q4 [3 marks] Explain why a pipeline may increase throughput without making a single request faster.

Computational methods: marking points

Check only after completing the independent questions.

Q1 · 4 marks — Abstraction versus decomposition

- Abstraction retains only features relevant to a sports app’s purpose. [1]
- For instance, a score tracker may omit shirt colour but retain scores. [1]
- Decomposition splits the app into smaller tasks. [1]
- For example, input, scoring rules and display may be implemented as separate parts. [1]

Q2 · 4 marks — Divide and conquer versus backtracking

- Divide and conquer divides a problem into smaller versions of the same task. [1]
- It solves those subproblems and combines their results. [1]
- Backtracking tries possible decisions to build a solution. [1]
- It reverses/abandons choices that cannot lead to a valid solution. [1]

Q3 · 4 marks — Benefit and limitation of a heuristic

- A heuristic can direct the search towards promising possibilities. [1]
- This can reduce the work compared with exhaustive exploration. [1]
- Its estimate can be misleading, so the first answer may not be optimal. [1]
- For example, a route with the cheapest next edge may be more costly overall. [1]

Q4 · 3 marks — Why pipelining raises throughput

- Different jobs can occupy different pipeline stages simultaneously. [1]
- Once filled, the pipeline can finish jobs more often than a non-overlapped process. [1]
- Each individual job still passes through all its required stages; latency need not fall. [1]

Equivalent technically correct responses can earn credit where they meet the stated criteria.

2.2.2 / ALGORITHMS AND PROGRAMMING

Computational methods

Computational methods help you turn an unclear real-world task into a precise solvable problem, then choose methods that work within time, memory and data-quality constraints.

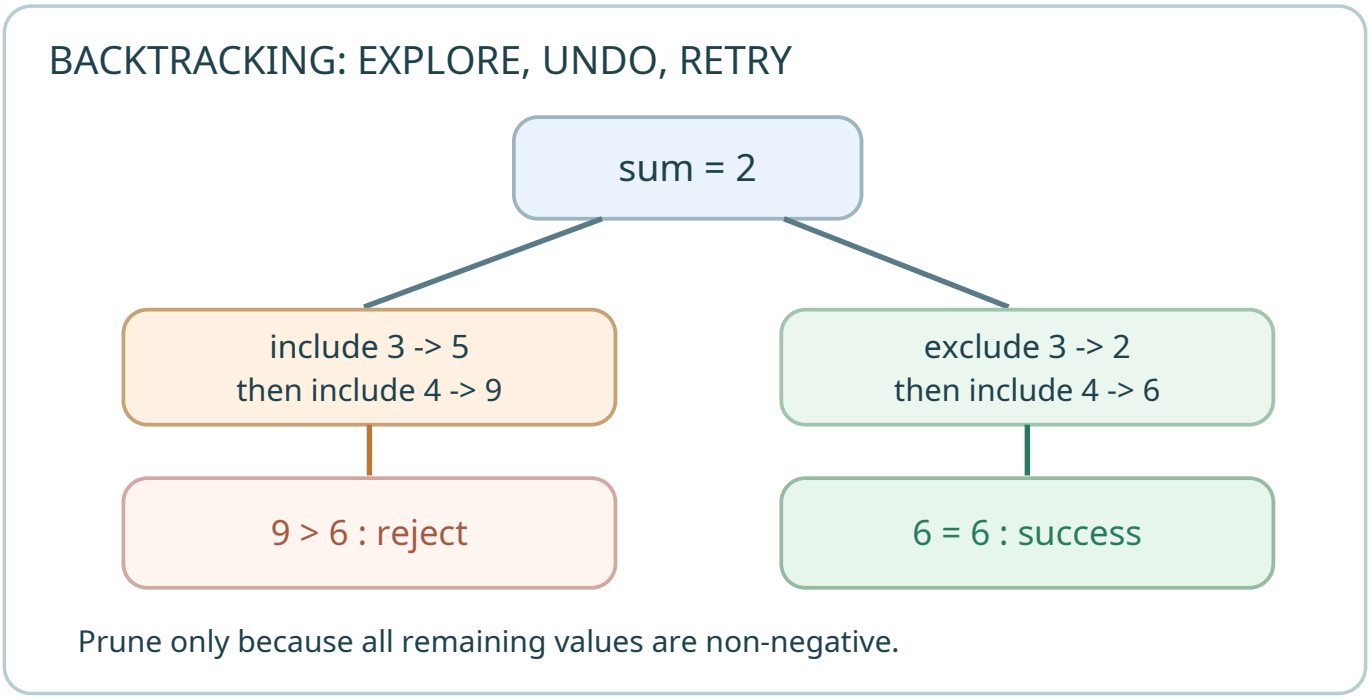
REMEMBER THIS

Backtracking is exploring a maze: choose a corridor, spot a dead end, retrace and try another. Data mining is detective work: patterns are clues, not proof of cause.

NOW BE PRECISE

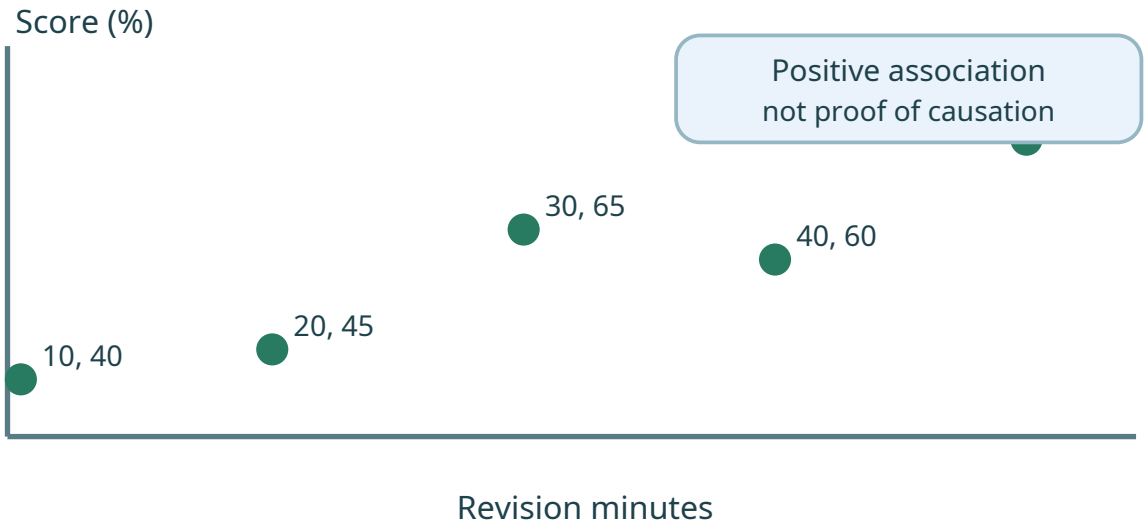
Problem recognition identifies a computable task; decomposition splits it; abstraction keeps relevant properties. Divide-and-conquer divides into subproblems. Heuristics guide search without guaranteeing an optimal answer. Performance modelling estimates resource use; pipelining overlaps stages to increase throughput.

Where the analogy stops: The maze only works as an analogy for systematically undoing choices. Pruning a sum just because it exceeds a target is only safe here when remaining values are non-negative.



Backtracking rejects an overshoot and explores another branch.

DATA MINING: A TREND IS NOT A CAUSE



A positive association does not establish a causal relationship.

PIPELINE: DIFFERENT JOBS USE DIFFERENT STAGES

	1	2	3	4	5	6
Fetch	Job 1	Job 2	Job 3	Job 4		
Decode		Job 1	Job 2	Job 3	Job 4	
Execute			Job 1	Job 2	Job 3	Job 4

Four jobs, three stages: ideal completion after 3 + 4 - 1 = 6 cycles.

Pipelining overlaps work from different jobs across stages.

WORKED EXAMPLE 1 / Find a subset by backtracking

```
Values: [2, 3, 4]   Target: 6
Try 2 + 3 + 4 = 9 -> reject
Backtrack and skip 3 -> 2 + 4 = 6
```

First attempt fails because 9 exceeds 6. Undo a choice and explore the exclude-3 branch; selecting 2 and 4 succeeds. The prune is justified by non-negative values.

WORKED EXAMPLE 2 / Different target

```
Values: [2, 4, 7]   Target: 9
Try 2 + 4 + 7, then backtrack.
```

The complete inclusion overshoots (13). Try excluding 4 and including 7 instead: 2 + 7 = 9. State which decision is undone and when you stop.

EXAM PRECISION

Another memory hook: a three-stage car wash represents pipelining. Different cars can occupy different stages simultaneously; a queue, a blocked station or a shared resource can reduce the ideal gain.

Exam practice

Q1 · 4 marks Explain the terms backtracking and pruning, with one reason a choice might be rejected.

Q2 · 6 marks Using non-negative values [1, 4, 6] and target 7, show a failed include-all branch and the successful subset found by backtracking.

Q3 · 6 marks Choose a bar chart, scatter chart or histogram for: comparing categories, comparing two numerical variables and showing frequencies in numerical intervals. Explain why correlation alone does not prove cause.

Q4 · 6 marks Four independent jobs pass through three one-cycle stages. Find the ideal pipelined completion time and compare it with sequential non-overlapped processing. Explain one reason real throughput may be lower.

Q5 · 6 marks A route planner uses a quick heuristic to prioritise roads. Explain two advantages of heuristics and two risks or limitations when modelling traffic conditions.

Model answers and marking points

Attempt the questions before checking your work. Equivalent technically accurate answers may also receive credit.

Q1 · 4 marks

Backtracking explores choices [1] and undoes them when necessary [1]. Pruning discards a branch [1] known unable to produce a valid or better result, for example an overshoot with non-negative remaining values [1].

Q2 · 6 marks

Include all 1+4+6=11 [1], greater than 7 [1], so that branch can be pruned [1]; undo the choice involving 4 [1], keep 1 and choose 6 [1], giving 7 [1].

Q3 · 6 marks

Bar chart for category comparisons [1]; scatter for paired numerical measurements [1]; histogram for interval frequencies [1]; association can be caused by confounding variables [1], coincidental sampling [1] or a reverse relationship [1].

Q4 · 6 marks

Pipeline fill takes three cycles [1]; each next job can start after one cycle [1]; finish after 3+4-1=6 cycles [2]; sequential requires 4×3=12 [1]; dependencies, resource contention or stalls can reduce gain [1].

Q5 · 6 marks

Can reduce the search effort [1] and find useful solutions quickly [1]. Estimates may be inaccurate or overestimate [1], so optimality is not automatic [1]. Real congestion weights change [1], and assumptions/data quality must be checked [1].

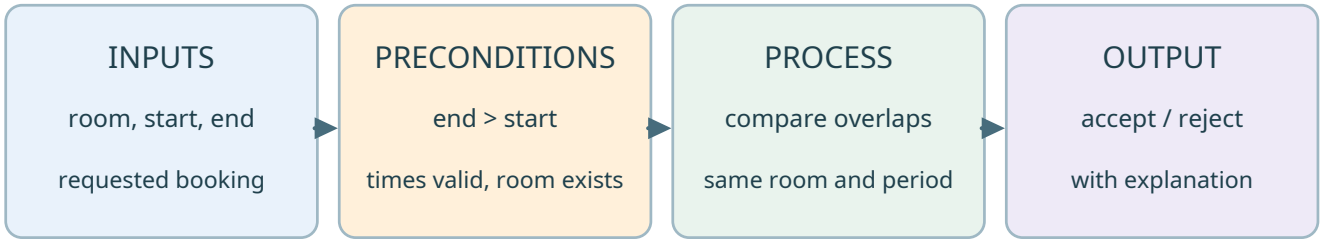
Define the computational problem

Recognise inputs, outputs, preconditions and boundaries before choosing an algorithm.

REMEMBER / THE RECIPE CARD

A precise computing problem needs **ingredients** (inputs), **rules** (preconditions), **steps** (algorithm) and a **result** (output). A recipe is only an analogy: a program must define its conditions and behaviour unambiguously.

FROM VAGUE TASK TO TESTABLE CONTRACT



EXAMPLE BOUNDARY: Half-open times [start, end):
09:00-10:00 and 10:00-11:00 do not overlap.

Make the acceptance rule unambiguous before writing code.

WORKED EXAMPLE 1 / SCHOOL ROOM BOOKING

Input: a room and proposed start/end times. Existing booking in that room is 09:00-10:00. New request is 10:00-11:00. Use **half-open intervals** [start,end), so touching endpoints do not overlap. The request is valid if start < end; it can be accepted if there is no conflicting existing booking.

General overlap test: **newStart < oldEnd AND oldStart < newEnd** (only when both bookings use the same room). For 10:00-11:00 and 09:00-10:00, the first comparison is false, so no overlap.

WORKED EXAMPLE 2 / DIFFERENT SYSTEM

An online library must approve a loan only when the **BookID** exists, the book is available, and the borrower is eligible. Identify the inputs, at least two preconditions and the output. Explain what to do if the BookID does not exist.

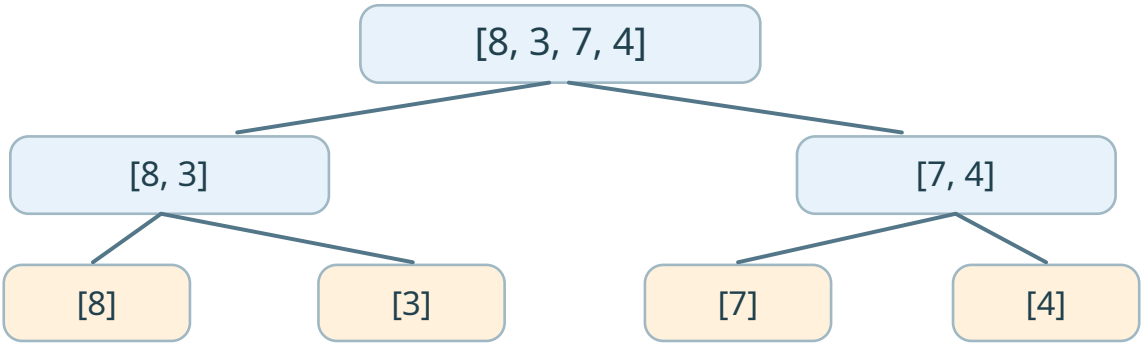
Divide and conquer

Solving smaller versions of the same problem and combining the answers.

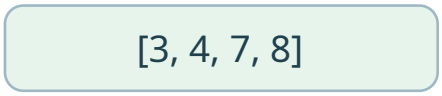
REMEMBER / SPLIT, SOLVE, REUNITE

Divide a list into smaller instances, solve each and combine. **Merge sort** really is divide and conquer; dividing work among unrelated functions is decomposition, but it is not automatically divide and conquer.

DIVIDE -> SOLVE -> COMBINE



Combine: [3,8] and [4,7]



Each half is sorted before they are merged.

WORKED EXAMPLE 1 / TRACE A DIFFERENT MERGE SORT

Sort **[7, 2, 6, 1]**. Split into [7,2] and [6,1]. Split again into singletons. Merge to [2,7] and [1,6]. Combine the sorted halves by comparing their front values: **[1,2,6,7]**.

WORKED EXAMPLE 2 / ANOTHER INPUT

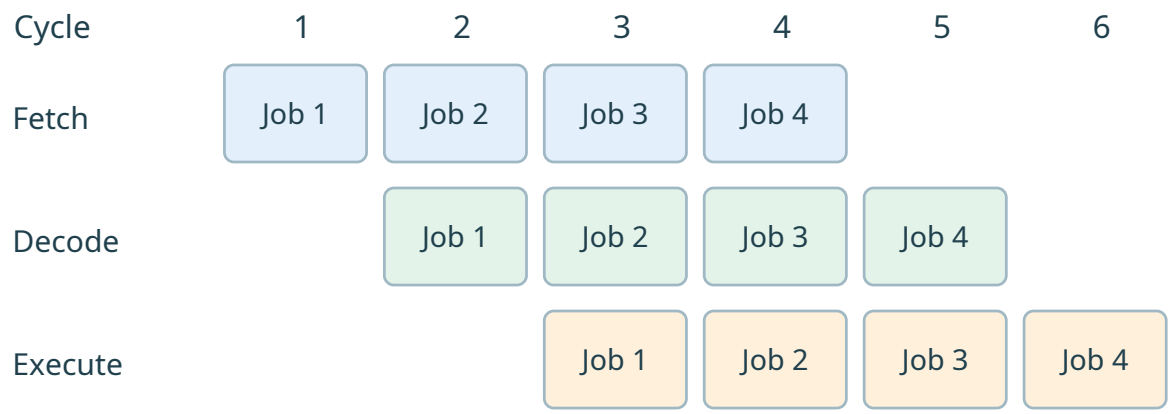
Trace **[9,5,8,3]** using the same approach. Include each two-element sorted run before the final merge.

Check: split [9,5] and [8,3], then [9], [5], [8], [3]. Merge to [5,9] and [3,8]. Finally merge to **[3,5,8,9]**.

Performance modelling and pipelining

Predict the benefit using stated assumptions, not an unsupported speed claim.

PIPELINING: OVERLAP THREE STAGES ACROSS SIX CYCLES



Ideal: four jobs, three equal stages = 3 + 4 - 1 = 6 cycles.

Without overlap: 4 x 3 = 12 stage-cycles.

WORKED EXAMPLE 1: Four independent jobs require Fetch, Decode and Execute stages, each taking one cycle. Without overlapping jobs: 4 x 3 = **12 cycles**. In an ideal balanced three-stage pipeline, total cycles = stages + jobs - 1 = 3 + 4 - 1 = **6 cycles**. This illustrates higher throughput, not necessarily shorter latency for one job.

WORKED EXAMPLE 2 / NEW WORKLOAD

Five jobs each pass through four equal one-cycle stages. Predict the ideal pipelined and sequential times, and calculate the saving. What happens if a stage stalls?

Check: Sequential = 5 x 4 = 20 cycles. Ideal pipeline = 4 + 5 - 1 = 8 cycles; saving = 12 cycles. A stalled stage or shared resource can delay following jobs, so actual time may be longer.

NOW BE PRECISE

A performance model simplifies reality using assumptions. Throughput is completed jobs per unit time; latency is the time a single job takes. Real constraints include bottlenecks, data dependencies, communication and pipeline stalls.

Design reusable solutions

Modularity supports testing and reuse, but does not automatically solve invalid inputs.

```
static bool Overlaps(int aStart, int aEnd,
                     int bStart, int bEnd) {
  return aStart < bEnd && bStart < aEnd;
}
static bool ValidTimes(int start, int end) {
  return start < end;
}
```

WORKED EXAMPLE 1: Overlaps(9,11,10,11) returns false, because 9 < 11 is true but 10 < 10 is false. ValidTimes(10,10) returns false. These functions can be tested independently and reused in a larger booking system.

WORKED EXAMPLE 2 / DIFFERENT PARAMETERS

Use the Overlaps function to decide whether 09:30-10:30 clashes with 10:00-11:00. The conditions are 9.5 < 11 and 10 < 10.5: both true, so the times overlap. In real code, represent times consistently as integer minutes or a suitable time type.

Common mistake: Never choose outputs or data types without defining the units. An integer minute count, for example 570 for 09:30, avoids ambiguity about decimal times.

Exam practice / problem recognition

Q1 [6 marks] A booking system must prevent two overlapping bookings in the same room. State two inputs, two preconditions and two outputs or outcomes that together define the computational problem.

Q2 [6 marks] Explain the difference between decomposition and divide and conquer using merge sort, and trace the two-element sorted runs when sorting [6,2,9,1].

Exam practice / performance modelling

Q3 [6 marks] Six independent jobs each pass through three equal one-cycle stages. Calculate sequential and ideal pipelined completion times and the ideal saving. Explain one assumption behind the prediction.

Q4 [6 marks] A school system estimates that 80% of a process can run in parallel while 20% must remain sequential. Explain why adding many processors cannot eliminate the sequential time, and discuss two further real-world constraints on speedup.

Exam practice / reusable components

Q5 [6 marks] Write C# method headers and return expressions for ValidTimes(start,end) and Overlaps(aStart,aEnd,bStart,bEnd). State the outcome for a proposed 10:00-11:00 booking compared with existing 09:00-10:00, assuming integer minutes.

Check your answers: computational methods

Check the calculations and the method conditions before self-marking.

Q1 · 6 marks

1. Give a room identifier as an input.
2. Give requested start/end times (or the current booking list) as inputs.
3. Require a valid/known room.
4. Require start before end and consistent time units.
5. Output a booking accepted/rejected decision.
6. Output an explanatory reason or a successfully recorded booking.

Q2 · 6 marks

1. Decomposition divides a task into manageable parts.
2. Divide and conquer breaks a problem into smaller instances of the same type.
3. It solves those subproblems and combines their results.
4. Split the array into **[6, 2]** and **[9, 1]**.
5. The sorted first two-element run is **[2, 6]**.
6. The sorted second run is **[1, 9]**.

Q3 · 6 marks

1. Sequential work is **6 × 3** stage-cycles.
2. Sequential total = **18 cycles**.
3. Ideal pipelining uses **3 + 6 − 1**.
4. Ideal total = **8 cycles**.
5. Ideal saving = **10 cycles**.
6. State an assumption: equal stage times, no stalls or independent jobs.

Q4 · 6 marks

1. Identify the non-parallel fraction as 20% of the work.
2. Explain that this portion still takes time regardless of extra processors.
3. Consequently speedup has a finite limit (ideal maximum **5×**, ignoring overhead).
4. Name one real-world constraint, such as communication or synchronisation overhead.
5. Explain how this constraint limits the speedup.
6. Identify and explain a second different constraint, such as memory bandwidth or load imbalance.

Q5 · 6 marks

1. ValidTimes takes integer start/end arguments and returns bool.
2. It returns the condition **start < end**.
3. Overlaps takes four integer times and returns bool.
4. It returns **aStart < bEnd && bStart < aEnd**.
5. Two bookings touching at 10:00 do not overlap under half-open intervals.
6. Therefore the proposed 10:00–11:00 booking is accepted against 09:00–10:00.

2.3.1 ALGORITHMS / Unfamiliar algorithms: trace, test and evaluate

REMEMBER / THE ALGORITHM'S CONTRACT

Before you trust a method, ask: What must be true on entry (precondition)? What remains true during a loop (invariant)? How will I test boundary cases? Then consider complexity.

DistinctCount in a SORTED array

[2, 2, 5, 5, 7, 7] -> 3 distinct values

index	0	1	2	3	4	5
value	2	2	5	5	7	7
count	1	1	2	2	3	3

Invariant: count = distinct groups seen so far.

WORKED EXAMPLE 1 / Counting distinct values

C# / MODEL

```
static int DistinctCount(int[] a) {
  if (a.Length==0) return 0;
  int count=1;
  for (int i=1; i<a.Length; i++)
    if (a[i]!=a[i-1]) count++;
  return count;
}
```

Preconditions and testing

Algorithm preconditions matter

SORTED

[1, 1, 2, 2, 4]

Correct result = 3

UNSORTED

[2, 1, 2]

False result = 3

The unsorted list has only 2 distinct values.

WORKED EXAMPLE 2 / change the example

Trace the sorted array [1,1,1,3,4,4]. Start at count = 1, then increase when the new value differs from its immediate predecessor.

GUIDED ANSWER

Values 1, 3 and 4 form three groups, so the return value is 3. For an empty array, the method returns zero immediately.

Test category	Input	Expected
Empty	[]	0
All equal	[5,5,5]	1
All different	[1,2,3]	3
Unsorted (invalid assumption)	[2,1,2]	Method gives 3, true answer is 2

EVALUATION / FIX

For unsorted data, sort first and then scan (typically O(n log n) time), or use a hash set to record values (expected O(n) time, O(n) extra memory). Be explicit about complexity assumptions.

Unfamiliar algorithms: trace, test and evaluate / exam practice

Q1 [6 marks] Trace DistinctCount on the sorted array [3,3,6,6,8,8]. Record count after each index and give the returned value.

WORKING / METHOD

Q2 [4 marks] Explain why DistinctCount assumes its array is sorted. Give a counterexample showing an incorrect result.

Unfamiliar algorithms: trace, test and evaluate / exam practice

Q3 [6 marks] Describe three tests for DistinctCount, including inputs and expected outputs. Explain why the empty-array test is necessary.

Q4 [7 marks] An unsorted input must be processed. Propose a correct algorithmic modification and compare the likely time and additional-space costs of sorting first versus using a hash set.

Unfamiliar algorithms: marking points

Check only after completing the independent questions.

Q1 · 6 marks

Trace DistinctCount

- At index 0, count starts at 1. [1]
- Index 1 has the same value: count=1. [1]
- Index 2 changes to 6: count=2. [1]
- Index 3 repeats 6: count=2. [1]
- Index 4 changes to 8: count=3. [1]
- Index 5 repeats 8: count=3; return 3. [1]

Q2 · 4 marks

Sorted precondition

- The method counts transitions between adjacent values. [1]
- Only sorted input guarantees that all copies of a value are consecutive. [1]
- For [2,1,2], the method would return 3. [1]
- The correct number of distinct values is 2. [1]

Q3 · 6 marks

Three test cases and the empty case

- Empty array [] with expected output 0. [1]
- All-equal array [5,5] with expected output 1. [1]
- All-distinct sorted array [1,2,3] with expected output 3. [1]
- The empty test checks that the special case is handled. [1]
- It prevents treating an empty collection as if it contained one distinct value. [1]
- It also guards against indexing a[0] when no first element exists, where relevant. [1]

Q4 · 7 marks

Repairing unsorted input

- One repair is to sort the input, then scan for adjacent changes. [1]
- A typical comparison sort takes O(n log n) time. [1]
- The following scan takes O(n) time. [1]
- Extra memory when sorting depends on the selected algorithm. [1]
- Alternatively, add each value to a hash set. [1]
- Expected time is O(n) with suitable hashing. [1]
- A hash set may need O(n) additional storage. [1]

Equivalent technically correct responses can earn credit where they meet the stated criteria.

2.3.1(a)-(d) / ALGORITHMS AND PROGRAMMING

Analysing and designing algorithms

An algorithm can give a correct answer and still be unsuitable for a large data set. Compare time, auxiliary memory, preconditions and performance on challenging inputs.

REMEMBER THIS

Time complexity is how the amount of work grows; auxiliary space is the extra luggage the algorithm carries. A short journey and a small suitcase are different advantages.

NOW BE PRECISE

A single full pass through n elements is O(n); a nested pair of complete n loops is $O(n^2)$; repeatedly halving the search interval is O(log n). Merge sort has O(n log n) worst-case time and commonly O(n) auxiliary space; standard in-place bubble sort is $O(n^2)$ worst-case and O(1) auxiliary space.

Where the analogy stops: Big O is a growth bound, not a stopwatch reading. Constants, implementation and typical inputs matter. Count input storage separately if the question asks specifically for auxiliary space.

BIG O: GROWTH, NOT A STOPWATCH

	n = 4	n = 8	n = 16
$O(\log_2 n)$	2	3	4
$O(n)$	4	8	16
$O(n \log_2 n)$	8	24	64
$O(n^2)$	16	64	256
$O(2^n)$	16	256	65,536

Illustrative operation counts with chosen simple models; not exact run times.

Illustrative growth counts for simple mathematical models; compare rate of growth, not execution time.

WORKED EXAMPLE 1 / Linear time and constant extra space

```
int sum = 0;
foreach (int number in values)
    sum += number;
```

The code visits n input elements once, so time is O(n). It uses one extra accumulator and loop control, so auxiliary space is O(1) (excluding the input array).

WORKED EXAMPLE 2 / Different loop pattern

```
for (int i=0; i<n; i++)
    for (int j=0; j<n; j++)
        comparisons++;
```

The inner increment executes n×n times: time $O(n^2)$. Only a fixed number of loop counters and an accumulator are stored: auxiliary space O(1).

EXAM PRECISION

Unfamiliar algorithms: before tracing, identify inputs, the required precondition, the loop invariant, the result and edge cases. For a distinct-value counter that compares adjacent elements, sorting is a necessary precondition.

Exam practice

Q1 · 4 marks Classify the dominant time complexity of: a constant-time lookup, one full pass, two nested full loops, and repeated halving.

Q2 · 6 marks Compare iterative and recursive binary search on a sorted array, including time complexity and auxiliary space.

Q3 · 6 marks A sorted list is [1,1,3,5,5,9]. A distinct-value counter starts at 1 and adds one whenever the current value differs from the preceding value. Trace the count and give the result.

Q4 · 8 marks The same adjacent-value counter is given unsorted input [2,1,2]. Explain the incorrect result, give two possible repairs, and compare their time and memory trade-offs.

Q5 · 6 marks Compare worst-case time and auxiliary space for an in-place bubble sort and a conventional merge sort, then justify when either might be a reasonable choice.

Model answers and marking points

Attempt the questions before checking your work. Equivalent technically accurate answers may also receive credit.

Q1 · 4 marks

Constant lookup O(1) [1]; full pass O(n) [1]; two complete nested loops $O(n^2)$ [1]; repeated halving O(log n) [1].

Q2 · 6 marks

Both repeatedly halve the search range [1] and take O(log n) worst-case time [2]. Standard iterative search uses O(1) auxiliary memory [1]; a straightforward recursive implementation has O(log n) call-stack usage [1]; both require sorted input for the standard algorithm [1].

Q3 · 6 marks

Start 1 [1]; next 1 no increment [1]; new 3 increases to 2 [1]; new 5 increases to 3 [1]; repeat 5 keeps 3 [1]; new 9 increases to 4 [1]. Result 4.

Q4 · 8 marks

Counter detects three runs and reports 3 [2], but there are only two distinct values [1]. Repair 1: sort then scan [1], taking typical O(n log n) time [1], with space dependent on sorting method [1]. Repair 2: insert into a hash set [1], expected O(n) time but O(n) extra space [1].

Q5 · 6 marks

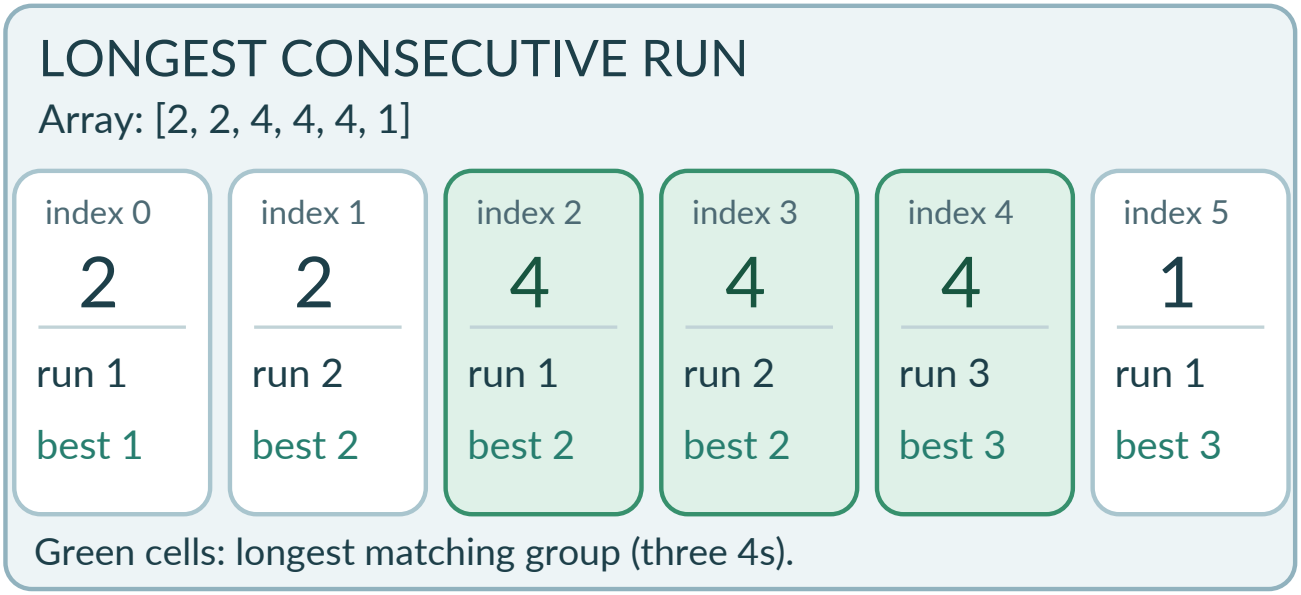
Bubble sort $O(n^2)$ worst case [1], typically O(1) extra space [1]; merge sort O(n log n) worst case [1], conventionally O(n) additional memory [1]; merge generally more suitable at large n [1], bubble can be simple for very small teaching examples or constrained extra memory [1].

Unfamiliar Algorithms: Trace, Test, Explain

In an unfamiliar algorithm, the challenge is to make the method visible: input, precondition, state, result, invariant, complexity and tests.

WORKED EXAMPLE 1 · Longest run of equal values

```
static int LongestRun(int[] a) {
    if (a.Length == 0) return 0;
    int current = 1, best = 1;
    for (int i = 1; i < a.Length; i++) {
        if (a[i] == a[i - 1]) current++;
        else current = 1;
        if (current > best) best = current;
    }
    return best;
}
```



For {2,2,4,4,4,1}, the longest adjacent run is three 4s. Invariant: current tracks the length of the run ending at index i; best tracks the largest run seen in the processed prefix.

Worst-case time O(n); auxiliary space O(1). Test empty, one element, all equal and alternating values.

WORKED EXAMPLE 2: a genuinely different algorithm

Find the first index where values increase compared with the previous element. Do not assume the array is sorted.

```
static int FirstRise(int[] a) {
    for (int i = 1; i < a.Length; i++) {
        if (a[i] > a[i - 1]) return i;
    }
    return -1;
}
// Input: {5, 4, 4, 7, 2}
```

At i=1, 4 is not greater than 5; at i=2, 4 is not greater than 4; at i=3, 7 > 4, so return 3. If there is no increase, return -1.

A reliable unfamiliar-algorithm checklist

- Input and output: what exactly is being returned?
- Preconditions: sorted input, non-null arrays, positive weights?
- State: list variables that change each step.
- Invariant: what remains true after each iteration?
- Correctness: show why the output answers the question.
- Efficiency and tests: time, additional memory and edge cases.

Remember this

TRACE: Test the assumption, Record each change, Apply the condition, Check the final result, Explain why it works. The phrase is a memory aid; use algorithmic terminology in written answers.

2.3.1 · EXAM PRACTICE

Trace and reason

The questions are independent of the WORKED EXAMPLE 1 and WORKED EXAMPLE 2 examples.

Q1 · 6 marks Trace current and best for the input {4,4,2,2,2,5}, and state the returned value. Use the LongestRun algorithm from this chapter.

Q2 · 5 marks For the FirstRise algorithm, trace {9,9,7,10,8} and state the returned index. Explain why the loop begins at index 1 and when -1 is returned.

2.3.1 · EXAM PRACTICE

Design from a specification

Ensure the precondition and edge cases are handled.

Q3 · 8 marks Write static int CountDistinctSorted(int[] a) for a non-null sorted integer array. Return 0 for an empty array, otherwise count distinct adjacent values. For {1,1,3,3,8}, return 3.

Q4 · 6 marks For LongestRun, justify the worst-case time and auxiliary-space complexity, state a loop invariant, and name two edge-case tests with their expected results.

2.3.1 · EXAM PRACTICE

Debug an unfamiliar search

The following method attempts binary search on an ascending sorted array. Find the defect and explain a failing case.

Q5 · 8 marks Identify the faulty update, repair it, show how searching for 2 in {3} can fail to progress in the original code, and state a precondition and worst-case time complexity.

```
static int Find(int[] a, int target) {
    int low = 0, high = a.Length - 1;
    while (low <= high) {
        int mid = low + (high - low) / 2;
        if (a[mid] == target) return mid;
        if (a[mid] < target) low = mid + 1;
        else high = mid;  // inspect this line
    }
    return -1;
}
```

2.3.1 · CHECK AND IMPROVE

Unfamiliar algorithms: marking points

Use complete algorithm states and avoid claiming a runtime from the test input alone.

Q1 · 6 marks

- Index 0: current=1,best=1 [1]; index 1: current=2,best=2 [1].
- Index 2: current=1,best=2 [1]; index 3: current=2,best=2 [1].
- Index 4: current=3,best=3 [1]; index 5: current=1,best=3, returning 3 [1].

Q2 · 5 marks

- Compare i=1 (9 vs 9) and i=2 (7 vs 9): neither rises [1].
- At i=3, 10 > 7 [1], so returns 3 [1].
- i starts at 1 to allow a[i-1] safely [1].
- -1 is returned when there is no rising adjacent pair [1].

Q3 · 8 marks

- Correct signature/return type [1]; empty-array check returning 0 [1].
- Initial distinct count 1 for nonempty input [1].
- Loop i from 1 to length-1 [2]; compare a[i] with a[i-1] [2].
- Increment when different and return the final count [1].

```
static int CountDistinctSorted(int[] a) {
    if (a.Length == 0) return 0;
    int count = 1;
    for (int i = 1; i < a.Length; i++)
        if (a[i] != a[i - 1]) count++;
    return count;
}
```

2.3.1 · CHECK AND IMPROVE

Unfamiliar algorithms: continued

The strongest answers link each line of code to a requirement in the problem.

Q4 · 6 marks

- Each element is processed once, hence O(n) time [1]; work per iteration is constant [1].
- Only current, best and a loop index are stored, hence O(1) auxiliary space [1].
- Invariant: current is the run ending at the latest index and best is the maximum within the prefix [1].
- Two correctly specified edge tests with outputs, e.g. [] → 0, [5] → 1, [2,2] → 2 [2].

Q5 · 8 marks

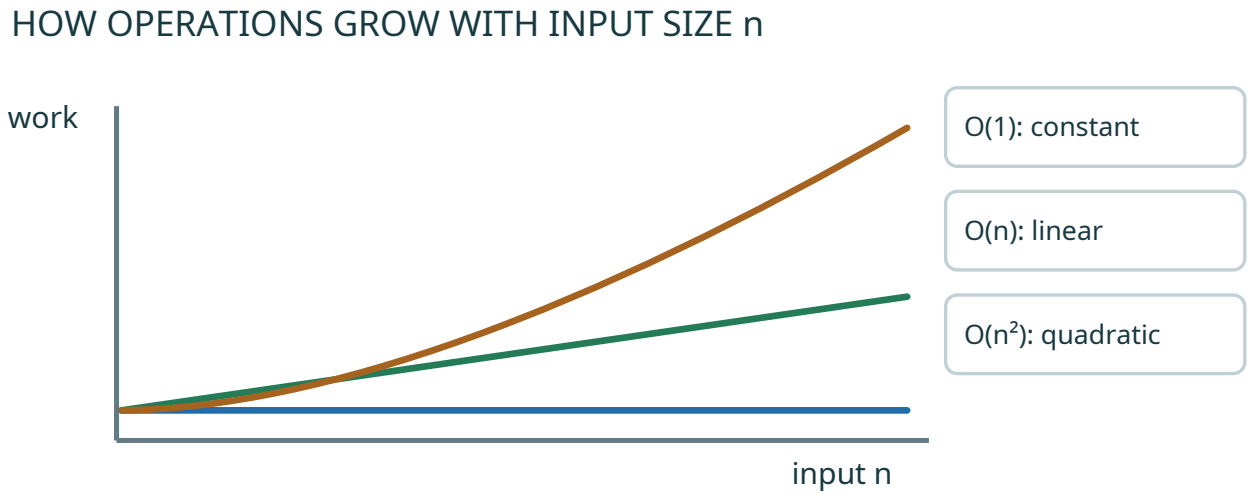
- The branch for a[mid] > target fails to reduce high [1].
- Correct it to high = mid - 1 [2].
- For a={3}, target=2: low=high=mid=0, else high=0 repeats indefinitely [2].
- The input must be sorted ascending [1].
- Each iteration discards about half of the search interval, so worst-case O(log n) [2].

Memory bank

Precondition: a condition the input must satisfy. Invariant: a property maintained throughout iteration. Complexity: how time or memory grows with input size. Test: an input with a known expected result.

2.3.1(C,D) / Big O: growth and complexity

Remember: Big O = how the workload grows as the input grows.



ANALOGY - PICTURE IT

One cashier for n customers suggests linear growth; comparing every customer with every other suggests roughly quadratic growth.

NOW BE PRECISE - WHERE THE ANALOGY STOPS

Big O is an asymptotic upper bound on growth, not the exact runtime in seconds. Hardware, constant factors and actual input patterns still matter.

THEORY / KEY TERMINOLOGY

O(1)	Bounded number of operations independent of n.
O(log n)	Problem size is repeatedly reduced by a constant factor.
O(n)	Work increases proportionally with n.
O(n²)	Nested work may scale with the square of n.
O(2^n)	Work can double when n increases by one.
Space complexity	How auxiliary storage requirement grows with input size.

Big O: growth and complexity / understand and model

OCR requires constant, logarithmic, linear, polynomial and exponential growth, plus comparison of time and space. Big O ignores constant multipliers and lower-order terms when describing the dominant asymptotic rate. Binary search is O(log n) when data is sorted and random indexing is available. A single pass over n items is O(n); two nested loops each running n times give $O(n^2)$. An algorithm may trade more memory for faster access, and worst-case time does not always match average-case behaviour.

WORKED EXAMPLE 1 / worked example

Algorithm: for i=0..n−1, for j=0..n−1, increment a counter.

Trace with n=4: the outer loop runs 4 times; each iteration performs 4 inner operations.

Count: 4 × 4 = 16 body executions.

Generalise: n × n = n^2; time complexity $O(n^2)$.

WORKED EXAMPLE 2 / same skill, different problem

YOUR TURN

Two nested loops each run exactly 5 times. How many times does the inner statement execute? What happens if n doubles from 5 to 10?

Scaffold: Generalise first to n × n. Then substitute 5 and 10.

My working:

GUIDED CHECK - READ AFTER ATTEMPTING

n=5: 25 operations.
n=10: 100 operations.
Scaling: Twice n gives four times the body executions.

Big O: growth and complexity · independent exam practice

Q1 [2 marks] Explain why a direct array index lookup is commonly O(1).

Q2 [4 marks] Contrast O(n) and O(log n) for growing inputs.

Q3 [3 marks] Identify the complexity of two nested loops each running n times and justify.

Big O: growth and complexity · independent exam practice

Q4 [3 marks] Explain why an O(n log n) sorting algorithm may require additional space.

Check your answers — Big O

Marks reward both the correct complexity statement and a sound reason.

Q1 · 2 marks · Array index lookup

- Random access uses the supplied index to locate an element. [1]
- Its number of steps does not grow with array length n: O(1). [1]

Q2 · 4 marks · O(n) versus O(log n)

- For linear time O(n), work grows proportionally with input size. [1]
- For logarithmic time O(log n), work grows by repeated reduction/halving. [1]
- Doubling input typically doubles a linear scan’s work. [1]
- Doubling input adds only about one halving step to a logarithmic process. [1]

Q3 · 3 marks · Nested loops

- The outer loop executes n times. [1]
- For each outer iteration, the inner loop executes n times. [1]
- There are $n \times n = n^2$ operations; time complexity is $O(n^2)$. [1]

Q4 · 3 marks · Extra memory for sorting

- Some sorts allocate a temporary array or list during merging. [1]
- That temporary structure consumes extra memory as input size grows. [1]
- For example, merge sort typically uses O(n) auxiliary space even though its time is O(n log n). [1]

Exam precision

Time complexity and auxiliary-space complexity are separate properties. Do not infer one from the other.

Stack algorithms: push and pop

LIFO: the most recently pushed remaining item is removed first.
Convention: stack[0..capacity-1]; top is the last occupied index.
Initially top = -1. Empty: top = -1. Full: top = capacity - 1.

WRITE THE ALGORITHM / Check before accessing the array

```
procedure push(value)
  if top == capacity - 1 then
    error("overflow")
  else
    top = top + 1
    stack[top] = value
  endif
endprocedure
```

```
function pop()
  if top == -1 then
    error("underflow")
  else
    result = stack[top]
    top = top - 1
    return result
  endif
endfunction
```

Algorithmic pseudocode: error reports failure and stops that operation.
State variables are shared by these operations. A failed operation changes no state.
Peek: after the empty check, return stack[top] without changing top.

WORKED EXAMPLE 1 / Trace a stack of capacity 3

Operation	top	Returned	Logical contents: bottom to top
initial	-1	-	empty
push(4)	0	-	4
push(9)	1	-	4, 9
pop()	0	9	4
push(7)	1	-	4, 7
peek()	1	7	4, 7

Pop reads the old top before decrementing it. Do not decrement first.
Removed values may remain physically in the array, but are outside the active stack.
Push and pop are O(1) for this fixed-capacity representation; no items are shifted.

WORKED EXAMPLE 2 / Try the next operations

Continue with push(2), then push(6), then pop(). Give the state after each.
Check: push(2) gives top = 2 and contents 4, 7, 2. push(6) reports overflow;
the state stays unchanged. pop() returns 2, leaving top = 1 and contents 4, 7.
Empty pop/peek reports underflow. Handling these cases prevents invalid access.

Theory / Key terminology

Use these exam-accurate terms when explaining algorithms and programming.

LIFO	Last In, First Out: the most recent stack item is removed first.
FIFO	First In, First Out: the earliest queued item is removed first.
Push and pop	Operations that add and remove items at the top of a stack.
Enqueue and dequeue	Operations that add at a queue's rear and remove at its front.
Circular queue	A fixed-size queue whose indices wrap around when necessary.
Binary search tree	Tree ordered so smaller keys appear left and larger keys right.
Traversal	Visiting nodes or vertices in a defined sequence.
Adjacency list	Graph representation storing neighbouring vertices for each vertex.

Queue algorithms: reuse the slots

FIFO: enqueue adds at the rear; dequeue removes from the front.
Lesson 020 starts with a linear queue. This circular version reuses freed slots.
Convention: front = oldest index; rear = newest index; count = occupied slots.
Initially front = 0, rear = -1, count = 0. Empty: count = 0; full: count = capacity.

COUNT-BASED CIRCULAR QUEUE / capacity must be positive

```
procedure enqueue(value)
  if count == capacity then
    error("overflow")
  else
    rear = (rear + 1) MOD capacity
    queue[rear] = value
    count = count + 1
  endif
endprocedure
```

```
function dequeue()
  if count == 0 then
    error("underflow")
  else
    result = queue[front]
    front = (front + 1) MOD capacity
    count = count - 1
    return result
  endif
endfunction
```

Algorithmic pseudocode: shared state; error stops the operation without changing it.
Peek: after the empty check, return queue[front] without changing front or count.

WORKED EXAMPLE 1 / Capacity 3: watch rear wrap from 2 to 0

Operation	front	rear	count	Returned	FIFO order
initial	0	-1	0	-	empty
enqueue(A)	0	0	1	-	A
enqueue(B)	0	1	2	-	A, B
dequeue()	1	1	1	A	B
enqueue(C)	1	2	2	-	B, C
enqueue(D)	1	0	3	-	B, C, D

Now queue[0]=D, queue[1]=B, queue[2]=C. Physical array order is not FIFO order.
front=1 identifies B as next; rear=0 identifies D as newest. The queue is full.
No shifting or resizing: each operation is O(1). MOD performs wraparound.

WORKED EXAMPLE 2 / Continue from the full queue

Try enqueue(E); then dequeue(); then enqueue(E) again. Track all three counters.
Check: first enqueue reports overflow. Dequeue returns B: front=2, rear=0, count=2.
Next enqueue stores E at index 1: front=2, rear=1, count=3; FIFO order is C, D, E.
Use the stated convention: other valid queues use different rear/empty rules.

2.3.1 / Algorithms for data structures

Linked lists, stacks, queues and trees

REMEMBER THIS Picture a train of linked carriages: change the couplings in the right order or you lose the rest of the train.

INSERT 7 BETWEEN 4 AND 9



1 New node 7 points to 9.
2 Node 4 now points to 7.

Final: 4 7 9 12

NOW BE PRECISE A pointer-based linked list need not be contiguous; array-based representations can instead use indexes as links. Stack and Queue are abstract data structures.

KNOW IT PRECISELY

Linked-list insert. Set new.Next to the following node BEFORE changing predecessor.Next.

Linked-list removal. Update the predecessor link to skip the removed node; handle head removal separately.

Stack. Push adds to top; Pop removes the most recent item (LIFO).

Queue. Enqueue adds at rear; Dequeue removes from front (FIFO).

BST. Left descendant keys smaller; right descendant keys larger, using a declared duplicate policy.

Algorithms for data structures / Worked method

Worked examples and a different guided problem.

UNDERSTAND IT

- Start with 4 → 9 → 12. Insert 7 between 4 and 9: newNode.Next = node4.Next; then node4.Next = newNode. Result: 4 → 7 → 9 → 12.
- To remove 9, find the preceding node 7 and set node7.Next = node9.Next. Result: 4 → 7 → 12; preserve the rest of the chain.
- BST insertion example: root20, left10, right30. For 12: compare with20 (go left), then10 (go right), insert at empty child.

WORKED EXAMPLE 1 / STACK AND QUEUE

Input 4,7,9. Stack Push(4),Push(7),Push(9),Pop() returns9. Queue Enqueue(4),Enqueue(7),Enqueue(9),Dequeue() returns4.

C# OPERATIONS IN A CUSTOM NODE CLASS

C# / OCR-style code

```
class Node {
  public int Value;
  public Node? Next;
  public Node(int v, Node? next=null) { Value=v; Next=next; }
}
Node inserted = new(7, node4.Next);
node4.Next = inserted;
```

WORKED EXAMPLE 2 / REMOVE THE HEAD

For head → 3 → 5 → 8, remove 3 by setting head = head.Next. The new head is 5 and the remaining 5 → 8 list remains reachable.

Linked-list and BST algorithms

Trace the mechanism carefully; these examples are not questions to memorise.

CREATE, INSERT AND REMOVE USING A CUSTOM NODE

C# / OCR-style code

```
class Node {
  public int Value;
  public Node? Next;
  public Node(int v, Node? next = null) {
    Value = v; Next = next;
  }
}
Node inserted = new(7, node4.Next);
node4.Next = inserted;
node7.Next = node9.Next; // remove node 9
```

For head removal, head = head.Next. A BST search compares against the current key and follows either the left or right child until a match or null. For an in-order traversal, visit left, root, right.

IMPORTANT The custom Node.Next is writable. In .NET, LinkedListNode<T>.Next is read-only: do not assign to it directly.

Algorithms for data structures / Exam practice

Complete the questions independently. Use the space provided for your working.

Q1 4 MARKS

Starting from 4 → 9 → 12, explain the two pointer updates needed to insert 7 between 4 and 9, and state the final order.

Q2 5 MARKS

After pushing/enqueuing 2, 5 and 8, state the first value removed by each.
Explain both removal rules and state where new items are inserted.

Algorithms for data structures / Exam practice

Complete the questions independently. Use the space provided for your working.

Q3 6 MARKS

For a BST with root20, left10 and right30, show where 12 is inserted and explain the path followed. Then describe one way to traverse the structure.

Check your answers: data-structure algorithms

Credit correctly described pointer operations and valid traversal methods.

Q1 · 4 marks

1. Construct or prepare a new node containing 7.
2. Set newNode.Next to the existing successor containing 9.
3. Set the node containing 4 to point to the new node.
4. The sequence becomes **4 → 7 → 9 → 12**.

Q2 · 5 marks

1. Stack Pop returns **8**.
2. A stack removes the most recently pushed item: LIFO.
3. Queue Dequeue returns **2**.
4. A queue removes the earliest enqueued item: FIFO.
5. New items are inserted at the **stack top** and the **queue rear**.

Q3 · 6 marks

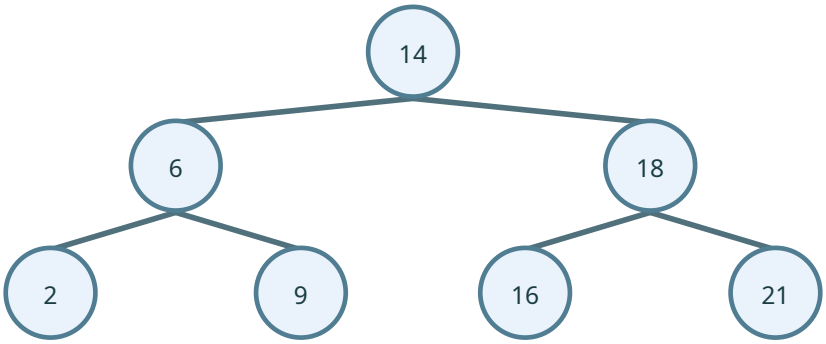
1. Compare 12 with root 20; 12 is smaller.
2. Follow the left reference to node 10.
3. Compare 12 with 10; 12 is larger.
4. Insert 12 as the (previously empty) right child of node 10.
5. Name a valid traversal, such as in-order, pre-order or post-order.
6. Describe the corresponding visit rule (e.g., **left → root → right** for in-order).

2.3.1 / TREE TRAVERSALS

REMEMBER THIS / EXPLORING A TREE

Post-order is LEFT → RIGHT → ROOT: finish the children before reporting the parent. Breadth-first is a level-by-level queue: visit nodes in order of discovery.

Same tree; different ways to visit its nodes



Post-order: 2, 9, 6, 16, 21, 18, 14

Level-order: 14, 6, 18, 2, 9, 16, 21

The same tree gives different output orders

Traversal	Rule	Result
Pre-order	Root, left, right	14, 6, 2, 9, 18, 16, 21
In-order	Left, root, right	2, 6, 9, 14, 16, 18, 21
Post-order	Left, right, root	2, 9, 6, 16, 21, 18, 14
Breadth-first	Levels, left child first	14, 6, 18, 2, 9, 16, 21

NOW BE PRECISE

Traversal is a systematic method of visiting nodes. Post-order is depth-first. Breadth-first traversal uses a FIFO queue. A stack would not preserve the same level order.

WORKED EXAMPLE 1 / TRACE THE QUEUE

Start from the lesson’s breadth-first algorithm: enqueue the root, then repeatedly dequeue a node, visit it and enqueue its left and right children.

Breadth-first search uses a FIFO queue

START -

Queue: 14

VISIT 14

Queue: 6, 18

VISIT 6

Queue: 18, 2, 9

VISIT 18

Queue: 2, 9, 16, 21

The next item removed is always the front of the queue.

WORKED EXAMPLE 1: complete queue trace

Step	Output so far	Queue (front first)
Initialise	(none)	14
Visit 14	14	6, 18
Visit 6	14, 6	18, 2, 9
Visit 18	14, 6, 18	2, 9, 16, 21
Visit 2	14, 6, 18, 2	9, 16, 21
Visit 9	14, 6, 18, 2, 9	16, 21
Visit 16	14, 6, 18, 2, 9, 16	21
Visit 21	14, 6, 18, 2, 9, 16, 21	empty

WORKED EXAMPLE 2 / a different tree

A tree has root 20; left child 10 (with right child 15) and right child 30. Trace breadth-first and post-order traversals.

GUIDED CHECK

Breadth-first: 20, 10, 30, 15. Post-order: 15, 10, 30, 20. Explain why 15 is visited at a different stage in the two methods.

REVISE / DFS VERSUS BFS

Key idea	Precise meaning
Root / leaf	The starting node of a tree / a node with no children.
Depth-first traversal	Explore subtrees before moving to siblings; post-order is an example.
Post-order	Left subtree, right subtree, then root.
Breadth-first traversal	Visit all nodes at one level before the next.
Queue	First In, First Out (FIFO), preserving the order nodes are discovered.
Stack	Last In, First Out (LIFO); suitable for some depth-first implementations.

Code first: breadth-first traversal in C#

```
var queue = new Queue<Node>();
queue.Enqueue(root);
while (queue.Count > 0)
{
    Node current = queue.Dequeue();
    Console.WriteLine(current.Key);
    if (current.Left != null) queue.Enqueue(current.Left);
    if (current.Right != null) queue.Enqueue(current.Right);
}
```

EXAM TIP

For a breadth-first trace, show both visited output and queue contents. For post-order, state the order as LEFT–RIGHT–ROOT and complete each subtree before its parent.

Tree traversals · independent exam practice

Q1 [3 marks] The tree has root 6, left child 3 and right child 9. Give the post-order traversal and explain why the root is last.

Q2 [5 marks] The tree has root 12; left child 5 has children 2 and 8, and right child is 17. Give breadth-first and post-order output sequences.

Tree traversals · independent exam practice

Q3 [6 marks] Describe a breadth-first algorithm using a queue. Explain why a stack is not equivalent in the usual left-to-right level-order traversal.

Q4 [6 marks] A tree has root 10, children 4 and 16, and 4 has children 1 and 7. Give pre-order, in-order and post-order traversals.

Tree traversals · independent exam practice

Q5 [4 marks] A programmer uses a stack instead of a queue for level-order traversal. Explain how the output can change, and identify a suitable fix.

Model answers: tree traversals

Check both the visit sequence and the rule governing the traversal.

Q1 · 3 marks

1. Post-order starts with left child **3**.
2. It visits right child **9**, then root **6**: **3, 9, 6**.
3. Explain that post-order processes the subtrees **before** visiting the root.

Q2 · 5 marks

1. Breadth-first starts at root 12, then level 2: 5, 17.
2. Breadth-first finishes level 3: 2, 8. Full output **12, 5, 17, 2, 8**.
3. Post-order visits the left subtree as **2, 8, 5**.
4. It then visits right child **17**.
5. It finishes at root **12**. Full output **2, 8, 5, 17, 12**.

Q3 · 6 marks

1. Enqueue the root.
2. Repeat while the queue is not empty.
3. Dequeue the front node.
4. Visit or output that node.
5. Enqueue its non-null children in left-to-right order.
6. A queue is FIFO and preserves level order; a stack uses LIFO and may descend a branch first.

Q4 · 6 marks

1–2. Pre-order: **10, 4, 1, 7, 16** (two credits for correct sequence/order).

3–4. In-order: **1, 4, 7, 10, 16** (two credits).

5–6. Post-order: **1, 7, 4, 16, 10** (two credits).

Q5 · 4 marks

1. A stack removes the most recently pushed node (LIFO).
2. It can visit a descendant before other nodes at the same level, changing the traversal order.
3. Use a queue instead.
4. FIFO ensures earlier-discovered nodes are processed before later-discovered nodes, preserving breadth-first order.

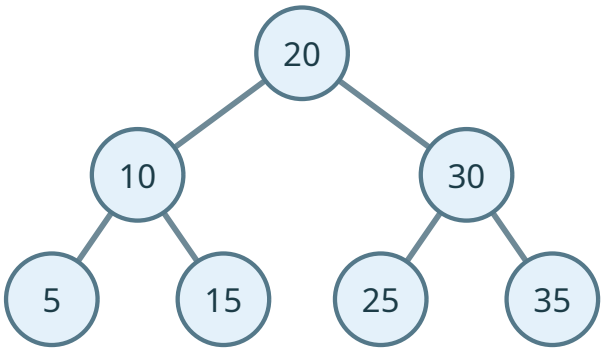
BST algorithms: insert, search and delete

Use key comparisons to trace the exact route, then update references safely.

REMEMBER THIS / SORTED SIGNPOSTS

At each BST node, smaller keys go **left** and larger keys go **right**. Deleting a node with two children needs a replacement key that preserves this ordering. The tree only remains a BST if the required links are updated.

BST DELETION: LEAF, ONE CHILD, TWO CHILDREN



In-order: 5, 10, 15, 20, 25, 30, 35

Leaf: remove 5; clear link
One child: reconnect the child

Two children: remove root 20
Choose in-order successor: 25
Replace root; delete old 25

New in-order: 5, 10, 15, 25, 30, 35

Assumption: this BST stores distinct integer keys. Deletion of a node with two children uses its in-order successor (smallest key in the right subtree); choosing the predecessor is also valid if done consistently.

WORKED EXAMPLE 1 / DELETE THE ROOT (TWO CHILDREN)

Original root 20; right subtree root 30 with left child 25. Find successor 25, copy its key to the root, then remove the old successor node from beneath 30. Final in-order: **5, 10, 15, 25, 30, 35**.

WORKED EXAMPLE 2 / A DIFFERENT INSERTION

Starting from root 20 with children 10 and 30, insert 28. Trace 28>20 (right), 28<30 (left). Add 28 as the left child of 30. Do not overwrite an existing child or lose a subtree.

Algorithms for BSTs

These short C# methods use a custom Node type; duplicate keys are ignored.

```
class Node {
  public int Key;
  public Node? Left, Right;
  public Node(int key) { Key = key; }
}
static Node Insert(Node? root, int key) {
  if (root == null) return new Node(key);
  if (key < root.Key) root.Left = Insert(root.Left,key);
  else if (key > root.Key) root.Right = Insert(root.Right,key);
  return root;
}
```

Search: compare key against current Key; if equal, found. If less, move to Left; if greater, move to Right; if null, not found. In-order traversal visits Left, Root, Right. For a BST with unique keys, it outputs ascending values.

DELETION CASES

Leaf: set the parent reference to null. **One child:** point the parent to the only child. **Two children:** find successor (leftmost right-subtree key), substitute its key, then delete the successor from its original location.

CHECK BEFORE YOU MOVE A LINK

In a linked list, save the old successor before changing the predecessor Next reference. In a BST, retain subtrees and propagate the updated child/root references after recursive insertion or deletion.

EXAM PRECISION / RECURSIVE INSERT

Recursive calls return the updated subtree root. Assign it back to root.Left or root.Right; at the top level use root = Insert(root, key), especially when the original tree is empty.

Exam practice: algorithms for trees

Q1 [4 marks] A BST has root 40, left 20 and right 60. Explain each comparison to insert key 50 and describe its final position.

Q2 [6 marks] A BST has keys 20, 10, 30, 5, 15, 25 and 35 as shown in the diagram. Delete root 20 using the in-order successor. Explain the link/key updates and give the final in-order traversal.

Exam practice: algorithms for trees

Q3 [6 marks] Describe an algorithm to search a BST for a key. Explain why a badly unbalanced BST can take linear time in the worst case.

Q4 [6 marks] Explain the three BST deletion cases (leaf, one child, two children). State one action needed to avoid losing part of a subtree.

Check your answers: BST algorithms

Q1 [4] 50>40 [1]; follow right branch to 60 [1]; 50<60 [1]; insert in empty left child of 60 [1].

Q2 [6] Successor is 25 [1]; copy 25 to root [1]; remove old 25 from left of 30 [1]; preserve right subtree 30 and 35 [1]; preserve 10 with 5 and 15 [1]; traversal 5,10,15,25,30,35 [1].

Q3 [6] Compare current node to target [1]; equal returns found [1]; less goes left [1]; greater goes right [1]; null means absent [1]; skewed tree may have n linked nodes on one path, O(n) [1].

Q4 [6] Leaf parent reference becomes null [1]; one-child parent points to child [1]; two children choose successor/predecessor [1], substitute key [1], remove original replacement node [1]; correct link propagation preserves remaining subtree [1].

2.3.1(f) / Linear and binary search

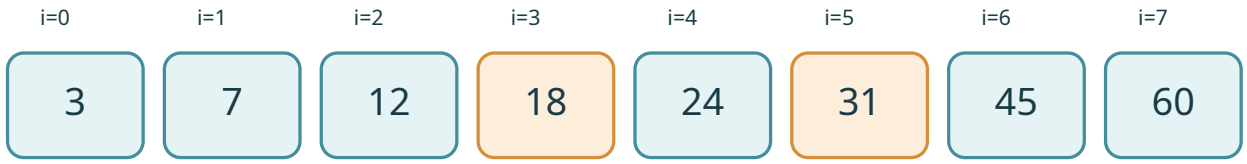
Begin with the sorted-array precondition and return an index, never just a value.

OCR 2.3.1(f) • SPECIFICATION CHECK

- Trace linear search and binary search with inclusive boundaries.
- Write a function that returns an index or −1 when absent.
- Compare O(n) and O(log n), and articulate the requirement for sorted data.

Binary search requires a sorted array.

Visual search for 31



mid = 3: value 18 is too small; mid = 5: value 31 is found.

Index 3 holds 18; since 31 > 18, search indices 4–7. New midpoint 5 contains 31: return index 5.

Feature	Linear search	Binary search
Precondition	Unsorted or sorted data.	Sorted by the comparison key.
Method	Check items sequentially.	Compare midpoint and discard half.
Worst-case time	O(n)	O(log n)
Found result	Return matching index.	Return matching index.
Absent	Return −1.	Return −1 when low > high.

Critical: Binary search can silently give an incorrect result on unsorted input. For an array of length n, the last index is n−1.

Theory / Key terminology

Use these exam-accurate terms when explaining algorithms and programming.

Linear search	Checks entries sequentially until a target is found or the list ends.
Binary search	Repeatedly halves a sorted search range.
Bubble sort	Repeatedly swaps adjacent out-of-order elements.
Insertion sort	Builds a sorted prefix by inserting each next value.
Merge sort	Divides a list and merges sorted portions.
Quick sort	Partitions around a pivot and sorts the partitions.
Time complexity	How algorithmic work grows as input size increases.
Sorted precondition	Binary search requires the input range to be ordered.

WORKED EXAMPLE 1 → WORKED EXAMPLE 2 / code then trace

WORKED EXAMPLE 1 / WRITE CORRECT BOUNDS AND MIDPOINT

```
static int BinarySearch(int[] a, int target)
{
    int low = 0, high = a.Length - 1;
    while (low <= high)
    {
        int mid = low + (high - low) / 2;
        if (a[mid] == target) return mid;
        if (a[mid] < target) low = mid + 1;
        else high = mid - 1;
    }
    return -1;
}
```

WORKED EXAMPLE 2 / trace three iterations

Trace target 15 in sorted [2, 5, 8, 12, 15, 20, 29]. Record bounds and midpoint.

Iteration	low	high	mid	a[mid]
1	_____	_____	_____	_____
2	_____	_____	_____	_____
3	_____	_____	_____	_____

Check: mid 3 gives 12 (low=4); mid 5 gives 20 (high=4);
mid 4 gives 15. Return index 4 after three comparisons.

Searching algorithms · independent practice

Q1 [4 marks] Trace binary search for 45 in [3, 7, 12, 18, 24, 31, 45, 60]. Show all midpoints.

WORKING / METHOD

Q2 [4 marks] Trace the same search for absent target 13. State the termination rule.

WORKING / METHOD

Q3 [3 marks] Explain why binary search is unsuitable on an unsorted collection of 1,000 values without preprocessing.

WORKING / METHOD

Searching algorithms · independent practice

Q4 [3 marks] Challenge: after 10 successful halvings, how large a search space could be reduced to at most one candidate? Explain.

Binary search: marking points

Check only after completing the independent questions.

Q1 · 4 marks

Find 45 in a sorted array

- Initial low=0, high=7: mid=3 and value 18. [1]
- Search upper half: mid=5 and value 31. [1]
- Search upper half again: mid=6 and value 45. [1]
- Return index 6 (zero-based). [1]

Q2 · 4 marks

Absent target 13

- Mid=3 gives 18, so set high=2. [1]
- Mid=1 gives 7, so set low=2. [1]
- Mid=2 gives 12, so set low=3. [1]
- low=3 exceeds high=2: target absent; return -1. [1]

Q3 · 3 marks

Why sorting is required

- Binary search decides which half to discard from the midpoint comparison. [1]
- This decision is reliable only if the array is sorted in the expected order. [1]
- For unsorted data, sort first or use a linear search instead. [1]

Q4 · 3 marks

Ten halvings

- A starting set of 2^10 = 1,024 candidates is possible. [1]
- Each halving reduces the remaining set to at most half its former size. [1]
- After ten halvings, at most one of those 1,024 candidates remains. [1]

Equivalent technically correct responses can earn credit where they meet the stated criteria.

2.3.1(f) / Sorting: trace, implement, evaluate

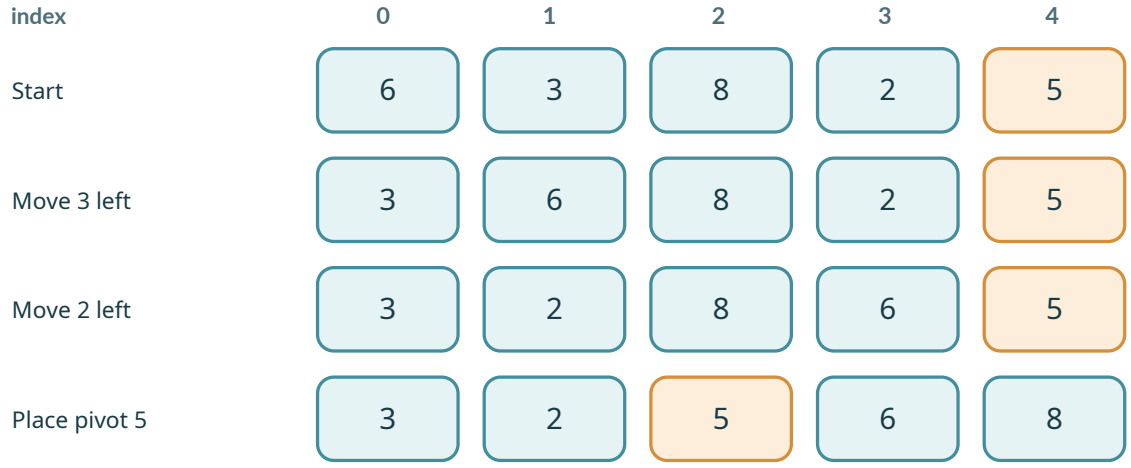
From comparison-and-swap to partitioning and merging; every intermediate state matters.

OCR 2.3.1(f) • SPECIFICATION CHECK

- Trace bubble and insertion sort; explain the stable sorted region.
- Divide, merge and combine sorted sublists.
- Partition around a pivot for quicksort and discuss performance.

Trace the array after each partition step.

Quicksort / a partition is not a full sort



Using the last element as pivot 5, [6,3,8,2,5] partitions to [3,2,5,6,8]. The pivot is in its final sorted position at index 2; the left partition [3,2] still needs sorting.

Algorithm	Working idea	Typical / worst case
Bubble	Adjacent compare-and-swap passes; largest unsorted item moves right.	$O(n^2)$ worst; optimised best O(n).
Insertion	Insert each new value in the correct position of a growing sorted prefix.	$O(n^2)$ worst; O(n) best on sorted input.
Merge	Divide, recursively/iteratively merge sorted halves.	O(n log n) time; extra space for merging.
Quick	Partition about a pivot and recursively sort each side.	O(n log n) average; $O(n^2)$ worst.

WORKED EXAMPLE 1 → WORKED EXAMPLE 2 / different sorts

WORKED EXAMPLE 1 / BUBBLE SORT - ONE COMPLETE PASS

As taught in Lesson 184: [6, 2, 5, 1] changes to [2, 5, 1, 6] after one left-to-right pass.

Comparison	Result
6 with 2	[2, 6, 5, 1]
6 with 5	[2, 5, 6, 1]
6 with 1	[2, 5, 1, 6]

WORKED EXAMPLE 2 / new insertion-sort array

Trace insertion sort on [7, 3, 6, 2] until the first three elements form a sorted prefix. Give the array after inserting 3 and after inserting 6.

Guided check: insert 3 into [7] → [3,7,6,2]; then insert 6 into [3,7] → [3,6,7,2]. Do not falsely claim the whole array is sorted.

Design question

Explain why merge sort needs additional storage when merging two temporary sorted lists.

Sorting — independent exam practice

Attempt each question before checking the separate marking points.

Q1 · 3 marks

Show the array after EACH adjacent swap during one complete left-to-right bubble-sort pass of [9, 4, 7, 1].

Q2 · 5 marks

Perform the first full quicksort partition of [8, 2, 6, 1, 4] using last-element pivot 4 and the Lomuto method. Give the pivot’s final index.

Q3 · 4 marks

Merge [2, 5, 9] and [1, 6, 7] into one ordered list. Explain the pointer/comparison method.

Sorting algorithms and Big O · independent practice

Q4 [4 marks] A programmer claims quicksort always takes O(n log n). Evaluate this claim.

Check your answers — sorting

Credit equivalent correct working. A complete pass and a complete sort are different things.

Q1 · 3 marks · One bubble-sort pass

- Swap 9 and 4: [4, 9, 7, 1]. [1]
- Swap 9 and 7: [4, 7, 9, 1]. [1]
- Swap 9 and 1: [4, 7, 1, 9]. [1]

Q2 · 5 marks · Lomuto partition

- Choose final pivot 4; initialise the left boundary. [1]
- Move 2 to index 0: [2, 8, 6, 1, 4]. [1]
- Move 1 to index 1: [2, 1, 6, 8, 4]. [1]
- Swap pivot into index 2: [2, 1, 4, 8, 6]. [1]
- The pivot’s final index is 2. [1]

Q3 · 4 marks · Merge

- Compare the front values of both sorted lists. [1]
- Append the smaller value and advance that list’s pointer. [1]
- Continue until one list is exhausted, then append the remainder. [1]
- Final merged result: [1, 2, 5, 6, 7, 9]. [1]

Q4 · 4 marks · Quicksort complexity

- Balanced partitions can give O(n log n) time. [1]
- Repeatedly extreme/imbalanced partitions can give $O(n^2)$ worst-case time. [1]
- Pivot selection/partitioning affects balance. [1]
- Therefore the claim “always O(n log n)” is false. [1]

2.3.1(f) / COMPLETE SORTING TRACES AND C#

Insertion, bottom-up merge and quicksort: the same output, three different sets of invariants.

REMEMBER THE JOBS

Insertion: sort playing cards into your hand. Merge: combine two already sorted queues. Quick: place a pivot in its final position, then sort the two sides. Each analogy helps only if you can trace the actual array.

Different sort, same goal: trace the intermediate state

INSERTION	7 3 6 2 -> 3 7 6 2 -> 3 6 7 2
MERGE	8 3 6 2 -> 3 8 | 2 6 -> 2 3 6 8
QUICK	8 1 7 3 6 -> 1 3 6 8 7 -> 1 3 6 7 8

Insertion grows a sorted prefix; merge combines sorted runs; quick fixes a pivot then sorts both sides.

WORKED EXAMPLE 1: insertion sort, complete pass sequence

Start [7, 3, 6, 2]. For each new key, shift larger values in the already sorted prefix one place right, then insert the key.

PASS	KEY	ARRAY AFTER PASS
Start	-	[7, 3, 6, 2]
1	3	[3, 7, 6, 2]
2	6	[3, 6, 7, 2]
3	2	[2, 3, 6, 7]

C# / OCR-style algorithm

```
static void InsertionSort(int[] a) {
    for (int i = 1; i < a.Length; i++) {
        int key = a[i], j = i - 1;
        while (j >= 0 && a[j] > key) {
            a[j + 1] = a[j];
            j--;
        }
        a[j + 1] = key;
    }
}
```

WORKED EXAMPLE 2: different data

Apply the same process to [5, 2, 4, 1]. After the three passes: [2,5,4,1] → [2,4,5,1] → [1,2,4,5]. Why is the left prefix sorted after each pass?

Bottom-up merge sort: no recursion necessary

Start [8,3,6,2]. Merge pairs into [3,8] and [2,6]. Merge those sorted runs: compare 3 with 2 (output 2), then 3 with 6 (output 2,3), then 8 with 6 (output 2,3,6); copy 8. Result [2,3,6,8]. This is bottom-up merge sort with run widths 1 then 2.

C# / OCR-style algorithm

```
static void MergeSortBottomUp(int[] a) {
    int n = a.Length;
    int[] tmp = new int[n];
    for (int width = 1; width < n; width *= 2) {
        for (int left = 0; left < n; left += 2 * width) {
            int mid = Math.Min(left + width, n);
            int end = Math.Min(left + 2 * width, n);
            int i = left, j = mid, k = left;
            while (i < mid && j < end)
                tmp[k++] = a[i] <= a[j] ? a[i++] : a[j++];
            while (i < mid) tmp[k++] = a[i++];
            while (j < end) tmp[k++] = a[j++];
        }
        Array.Copy(tmp, a, n);
    }
}
```

Quicksort: the partition is not the full sort

Use the final element as pivot (Lomuto partition). For [8,1,7,3,6], scan left to right. Move 1 left, then 3, then swap pivot 6 into index 2. Result [1,3,6,8,7]. The pivot is correctly positioned, but the right side still needs sorting.

C# / OCR-style algorithm

```
static int Partition(int[] a, int low, int high) {
    int pivot = a[high], i = low;
    for (int j = low; j < high; j++) {
        if (a[j] <= pivot) {
            (a[i], a[j]) = (a[j], a[i]);
            i++;
        }
    }
    (a[i], a[high]) = (a[high], a[i]);
    return i;
}
static void QuickSort(int[] a, int low, int high) {
    if (low >= high) return;
    int p = Partition(a, low, high);
    QuickSort(a, low, p - 1);
    QuickSort(a, p + 1, high);
}
```

EXAM PRECISION

Insertion worst $O(n^2)$, best $O(n)$ with the usual shifted-key algorithm. Bottom-up merge $O(n \log n)$ time, $O(n)$ auxiliary space. Quicksort average $O(n \log n)$, worst $O(n^2)$ with poor pivot splits; recursion requires stack space.

Independent practice

Q1 [5 marks]

Trace every insertion-sort pass on [6, 1, 5, 2]. Show the intermediate arrays and final sorted output.

Q2 [6 marks]

Perform bottom-up merge sort on [7, 4, 8, 3]. Show the width-1 sorted runs, the order of comparisons during the width-2 merge, and the final array.

Q3 [6 marks]

Partition [9, 2, 8, 3, 5] using the final element 5 and Lomuto partitioning. Show every swap, the pivot index, and which subarrays still need sorting.

Q4 [6 marks]

Compare the worst-case time and auxiliary-space requirements of insertion sort, bottom-up merge sort and quicksort. Explain one reason an implementation might favour one over another.

Check and improve: sorting marking points

Q1 [5 marks] Pass 1 [1,6,5,2] [1]; pass 2 [1,5,6,2] [1]; pass 3 [1,2,5,6] [2]; each key is inserted into an already sorted prefix [1].

Q2 [6 marks] Width 1: [4,7] and [3,8] [2]. Width 2 merge: compare 4 vs 3 → choose 3 [1]; 4 vs 8 → choose 4 [1]; 7 vs 8 → choose 7 [1]; copy 8 → [3,4,7,8] [1].

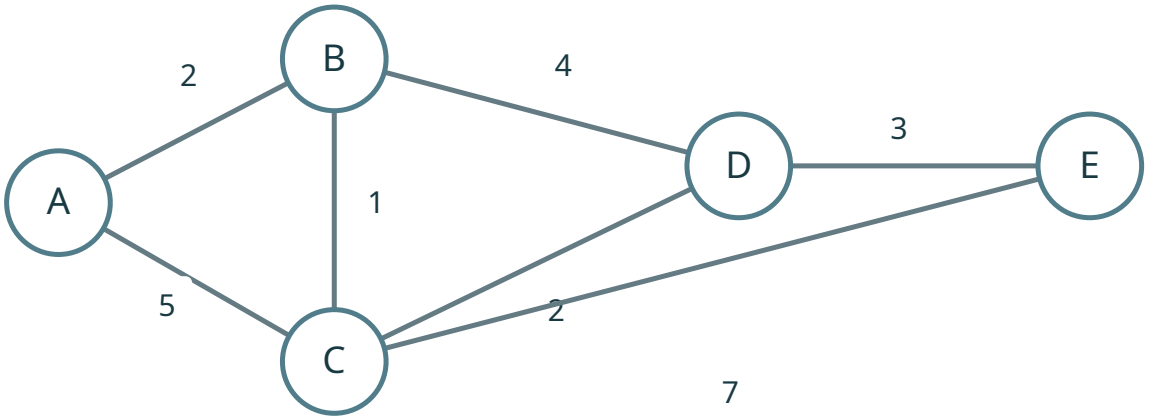
Q3 [6 marks] Start [9,2,8,3,5]; move 2 to index 0 → [2,9,8,3,5] [1]; move 3 to index 1 → [2,3,8,9,5] [1]; swap pivot into index 2 → [2,3,5,9,8] [2]. Pivot index 2 [1]; [2,3] and [9,8] require recursive handling (right is unsorted) [1].

Q4 [6 marks] Insertion worst $O(n^2)$ and O(1) extra [2]; bottom-up merge worst O(n log n) and O(n) extra [2]; quicksort worst $O(n^2)$ but average O(n log n), using call-stack space [1]; appropriate justification such as memory versus time or nearly sorted small data [1].

2.3.1(F) / Dijkstra and A* pathfinding

Remember: Dijkstra uses price tags: keep the cheapest known route so far.

DIJKSTRA: SETTLE LOWEST DISTANCE FIRST



Edges show non-negative costs; store predecessors to reconstruct the route.

ANALOGY - PICTURE IT

At each junction, attach a tentative total cost from Start. Settle the cheapest unsettled junction and update neighbours.

NOW BE PRECISE - WHERE THE ANALOGY STOPS

A tentative value is not always final until settled. Dijkstra requires non-negative edge weights; A* uses an estimated remaining cost as well.

THEORY / KEY TERMINOLOGY

Tentative distance	Best distance currently known from the start vertex.
Settled vertex	Vertex whose shortest distance is final under Dijkstra’s assumptions.
Relaxation	Replace a neighbour distance when a cheaper route is found.
Predecessor	Previous vertex stored to reconstruct the path.
A* heuristic h(n)	Estimated remaining cost from n to the goal.
f(n) = g(n)+h(n)	Estimated total route score used to prioritise A* exploration.

Dijkstra and A* pathfinding / understand and model

Dijkstra initialises Start at 0 and other distances at infinity, settles the lowest-distance unsettled vertex, and relaxes neighbouring edges. It works with non-negative weights. A* prioritises vertices using known cost g plus estimated remaining cost h. An admissible h never overestimates the true remaining cost; with appropriate conditions and bookkeeping, A* can find an optimal route. Both algorithms should preserve predecessors to reconstruct a route, not just its total weight.

WORKED EXAMPLE 1 / worked example

Graph: A–B 2, A–C 5, B–C 1, B–D 4, C–D 2, C–E 7, D–E 3.

Settle A=0: B=2, C=5. Settle B=2: C becomes 3; D becomes 6.

Settle C=3: D becomes 5; E becomes 10. Settle D=5: E becomes 8.

Answer: settle E=8. Reconstruct A→B→C→D→E with cost 2+1+2+3=8.

WORKED EXAMPLE 2 / same skill, different problem

YOUR TURN

A second graph has edges A–B 3, A–C 4, B–C 2, B–D 5, C–D 1 and D–E 2. Find the shortest route A to E using Dijkstra and show tentative distances.

Scaffold: Start with A=0 and infinity elsewhere. Settle A, then the smallest available tentative distance.

My working:

GUIDED CHECK - READ AFTER ATTEMPTING

Settle sequence: A(0), B(3), C(4), D(5), E(7).
Path: A–C–D–E.
Distance: 4 + 1 + 2 = 7.

Dijkstra and A* pathfinding · independent exam practice

Q1 [3 marks] Explain what it means to relax an edge in Dijkstra’s algorithm.

Q2 [3 marks] Why must standard Dijkstra have non-negative edge weights?

Q3 [3 marks] An A* node has g=8 and h=5. State f and explain g and h.

Dijkstra and A* pathfinding · independent exam practice

Q4 [4 marks] Challenge: explain why an admissible heuristic helps A* preserve shortest-path optimality under appropriate implementation conditions.

CODE / PSEUDOCODE

Dijkstra and A*: marking points

Check only after completing the independent questions.

Q1 · 3 marks — Relaxing an edge

- Calculate a candidate distance via the current node: dist[u] + weight(u,v). [1]
- Compare it with the current tentative distance of v. [1]
- If smaller, update v’s distance (and predecessor for path reconstruction). [1]

Q2 · 3 marks — Non-negative weights

- Dijkstra permanently settles the currently smallest tentative distance. [1]
- With a negative edge, a path discovered later could reduce a previously settled distance. [1]
- That breaks the correctness assumption of the standard greedy algorithm. [1]

Q3 · 3 marks — A* score

- f = g + h = 8 + 5 = 13. [1]
- g is the known path cost from the start to this node. [1]
- h is the heuristic estimate of the remaining cost to the goal. [1]

Q4 · 4 marks — Admissible heuristic and optimality

- An admissible heuristic does not overestimate the true minimum remaining cost. [1]
- A* ranks nodes by f=g+h, combining cost so far with the estimate. [1]
- The non-overestimating bound helps preserve a route to an optimal solution. [1]
- Correct termination and appropriate open/closed-set handling are also required; for some variants, inconsistent heuristics require reopening nodes. [1]

Equivalent technically correct responses can earn credit where they meet the stated criteria.

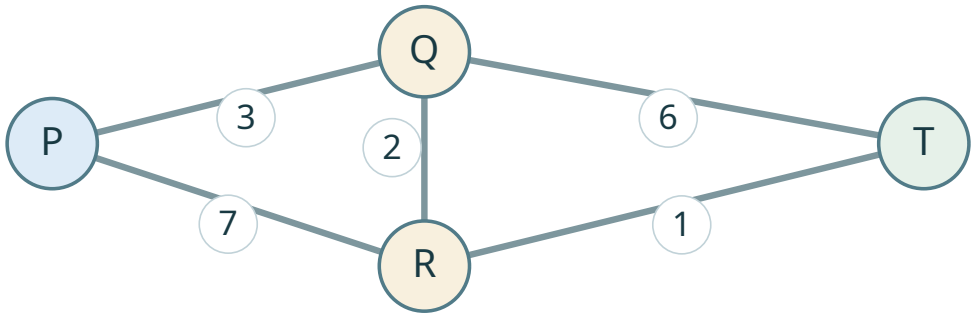
2.3.1(f) / DIJKSTRA AND A*: COMPLETE PATHFINDING

Both search weighted graphs, but A* additionally uses a heuristic estimate to the goal.

REMEMBER THE ROLES

Dijkstra expands the closest unsettled vertex by known distance. A* compares f = g + h: distance already travelled plus an estimated distance remaining. In both methods, always update predecessors when a better route is found.

Dijkstra: discover, relax, settle, reconstruct



All edges have non-negative weights. The numbers on lines are edge costs.

WORKED EXAMPLE 1: Dijkstra trace from P to T

Edges P–Q=3, P–R=7, Q–R=2, Q–T=6, R–T=1. All weights are non-negative. Initialise P=0, others infinity. Each row records distances after settling the named vertex.

SETTLED	P	Q	R	T
P	0	3	7	∞
Q	0	3	5	9
R	0	3	5	6
T	0	3	5	6

Predecessor updates: Q←P, R←Q and T←R. Reverse T←R←Q←P to obtain P–Q–R–T, cost 6. T is not final merely because it first receives distance 9; it becomes final when settled.

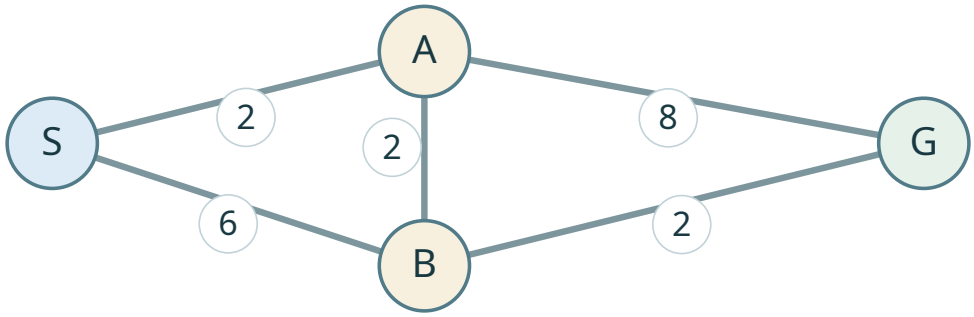
Dijkstra / clear algorithmic pseudocode

```
dist[start] = 0; every other dist = infinity
while an unsettled reachable vertex remains:
    u = unsettled vertex with smallest dist
    settle u
    for each edge (u, v, weight):
        if dist[u] + weight < dist[v]:
            dist[v] = dist[u] + weight
            predecessor[v] = u
```

WORKED EXAMPLE 2: different graph

M–N=4, M–O=2, O–N=1, N–T=3, O–T=8. After M: N=4, O=2. Settle O: improve N=3 and T=10. Settle N: improve T=6. Settle T: route M–O–N–T, cost 6.

A*: choose the open vertex with lowest f



All edges have non-negative weights. The numbers on lines are edge costs.

WORKED EXAMPLE 1: A* from S to G

Edges S–A=2, S–B=6, A–B=2, A–G=8, B–G=2. Use h(S)=5, h(A)=3, h(B)=1, h(G)=0. Here h is admissible; it never exceeds the true cheapest remaining cost.

A*: compare estimated total costs, not just h

VERTEX	g from start	h to goal	f = g + h
S	0	5	5
A	2	3	5
B	6	1	7

EXPAND	NEW / IMPROVED FRONTIER	SELECT NEXT
S	A: g2 h3 f5; B: g6 h1 f7	A (f5)
A	B: g4 h1 f5; G: g10 h0 f10	B (f5)
B	G: g6 h0 f6	G (f6)

Predecessors G←B←A←S yield S–A–B–G, cost 6. The algorithm compares lowest f, not lowest h alone. In this example the goal is selected with optimal cost using an admissible, consistent heuristic.

A* / conceptual pseudocode (reopen on improvement)

```
open = {start}; g[start] = 0; f[start] = h(start)
while open is not empty:
    u = open vertex with smallest f
    if u == goal: reconstruct predecessors; stop
    remove u from open; mark u closed
    for each neighbour v and edge weight w:
        candidate = g[u] + w
        if candidate improves g[v]:
            predecessor[v] = u; g[v] = candidate
            f[v] = g[v] + h(v); add/reopen v
```

NOW BE PRECISE

Admissible means h never overestimates the cheapest remaining cost. With graph-search details such as reopening improved vertices handled correctly, this supports an optimal path. Overestimating can cause an early-goal implementation to return a longer route.

Independent practice

Q1 [6 marks]

Use Dijkstra on edges M–N=4, M–O=2, O–N=1, N–T=3, O–T=8. Show the order settled, final distances and predecessor route from M to T.

Q2 [6 marks]

Use A* on edges U–V=2, U–W=6, V–W=2, V–G=8, W–G=2, with h(U)=5, h(V)=3, h(W)=1, h(G)=0. Record g, h and f for the frontier changes and give the final route.

Q3 [5 marks]

Explain why Dijkstra requires non-negative weights. Define an admissible A* heuristic, and explain the effect of a serious overestimate on an early-goal search.

Check and improve: pathfinding marking points

Q1 [6 marks] Settle M(0), O(2), N(3), T(6) [2]. Final distances M0, O2, N3, T6 [2]. Predecessors O←M, N←O, T←N; route M–O–N–T, cost 6 [2].

Q2 [6 marks] After U: V g2 h3 f5, W g6 h1 f7 [2]. After V: W improves to g4 h1 f5; G gets g10 h0 f10 [2]. After W: G improves to g6 h0 f6; route U–V–W–G, cost 6 [2].

Q3 [5 marks] With negative weights, a previously settled shortest distance may later become lower, invalidating Dijkstra’s greedy finalisation [2]. Admissible h does not overestimate the true remaining shortest distance [2]. A large overestimate can cause premature selection of a suboptimal goal path [1].

H446 MEMORY BANK

Read the hook, then cover the precise wording and try to explain the concept without the analogy.

Thinking / 2.1.1 / Bouncer checking guest list

NOW BE PRECISE

Abstraction keeps only fields relevant to one job.

Careful: A different purpose may require different fields.

SIS / 2.2.1 / Sequence, Iteration, Selection

NOW BE PRECISE

Order, repeat, decide (OCR: branching).

Careful: SIS is a memory aid, not official OCR shorthand.

Recursion / 2.2.1 / Nesting dolls

NOW BE PRECISE

Calls go deeper until base case; results unwind back up.

Careful: Every recursive call requires a termination path.

Search / 2.3.1 / Higher/lower game

NOW BE PRECISE

Binary search halves the remaining *sorted* search region.

Careful: Return index not just value; unordered data breaks it.

MEMORY BANK / continued

Sort / 2.3.1 / Tidying playing cards

NOW BE PRECISE

Insertion sort builds a sorted prefix; bubble compares neighbours.

Careful: Quicksort performance depends on pivot and partitioning.

Methods / 2.2.2 / Maze with breadcrumbs

NOW BE PRECISE

Try, test, undo, retry = backtracking.

Careful: A heuristic does not guarantee the best route.

Big O / 2.3.1 / Customers at checkouts

NOW BE PRECISE

Think about how work grows when n grows.

Careful: Not exact execution time in seconds.

Paths / 2.3.1 / Price tags

NOW BE PRECISE

Dijkstra settles the smallest tentative distance; A* uses g+h.

Careful: Dijkstra needs non-negative weights; h can affect A* search order.

REVISION ROUTINE

1. Recall the hook without looking. 2. Explain in precise H446 language. 3. Trace or draw an example. 4. Attempt a fresh exam question.

MEMORY BANK

Cover the precise meaning and explain each memory hook aloud. The analogy helps you remember; the technical explanation earns the marks.

FOUR MOVES	Keep / prepare / order / decide
EXAM PRECISION	Think abstractly, ahead, procedurally and logically.
CACHE	Fast but can be stale
EXAM PRECISION	A cache saves computation but needs refresh or invalidation.
BOOKING	Validate / check / save / confirm
EXAM PRECISION	Subprocedures must respect dependencies and failed operations.
SIS	Sequence / iteration / selection
EXAM PRECISION	Constructs occur together; OCR also uses branching.
RETURN VALUE	Capture the result
EXAM PRECISION	A returned value changes a caller variable only if assigned or otherwise used.
BY REF	An alias, not a copy
EXAM PRECISION	A ref parameter can update the original caller variable.
MODEL / REALITY	Fit to purpose
EXAM PRECISION	Abstractions preserve relevant details, not every feature.

Memory bank · additional recall

Cover the precise wording, recall the memory hook, then explain the computer-science meaning aloud.

Maze: undo and retry	Backtracking Explore choices, undo failed routes and systematically try alternatives.
Detective with a chart	Data mining Find patterns; correlation alone does not demonstrate causation.
Car wash	Pipelining Overlap stages on different items to improve throughput, allowing for stalls.
Extra luggage	Auxiliary space Memory used in addition to the original input data.
Bigger inputs	Big O Compare asymptotic growth, not exact running time.
Sorted shelf	DistinctCount Adjacent comparisons count distinct values correctly when equal values are together.